

курс «Глубокое обучение»

Эффективное обучение нейронных сетей **часть 2**

Александр Дьяконов

Борьба с переобучением / сложностью

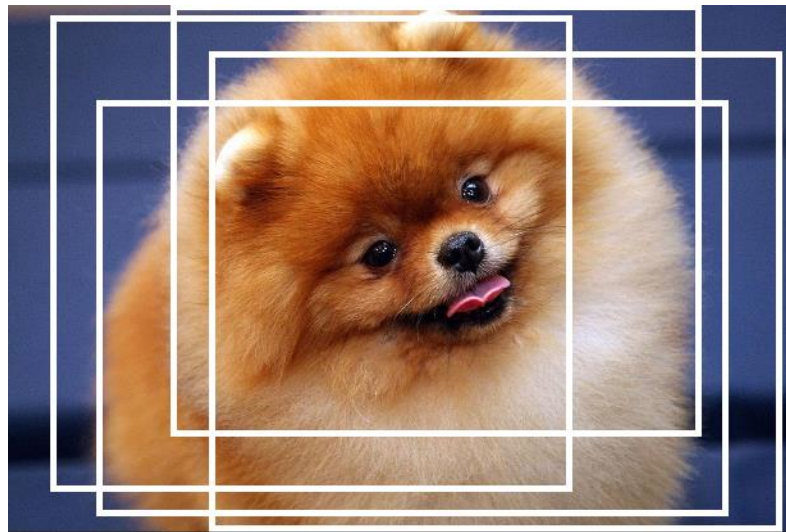
- Увеличение выборки + расширение выборки (Data Augmentation)
- Доучивание уже настроенных нейросетей (Pre-training, Transfer Learning)
- Ансамбль нейросетей

- Диагностика проблем с НС
- Настройка темпа обучения
- Оптимизация гиперпараметров
- Глупые ошибки

сейчас пока не будет

- Unsupervised Learning
- Техника зануления весов (разреживания НС)
- Использование специальных архитектур под задачу
- Разделение параметров (Parameter Sharing)

Расширение обучающего множества (Data Augmentation)



звук

- + фоновый шум
- тональность

текст

- замена синонимов
- перестановки, удаления слов
- смесь с другим текстом
- преобразования (ex: переводчик и обратно)

Аугментация – построение дополнительных данных из
ИСХОДНЫХ

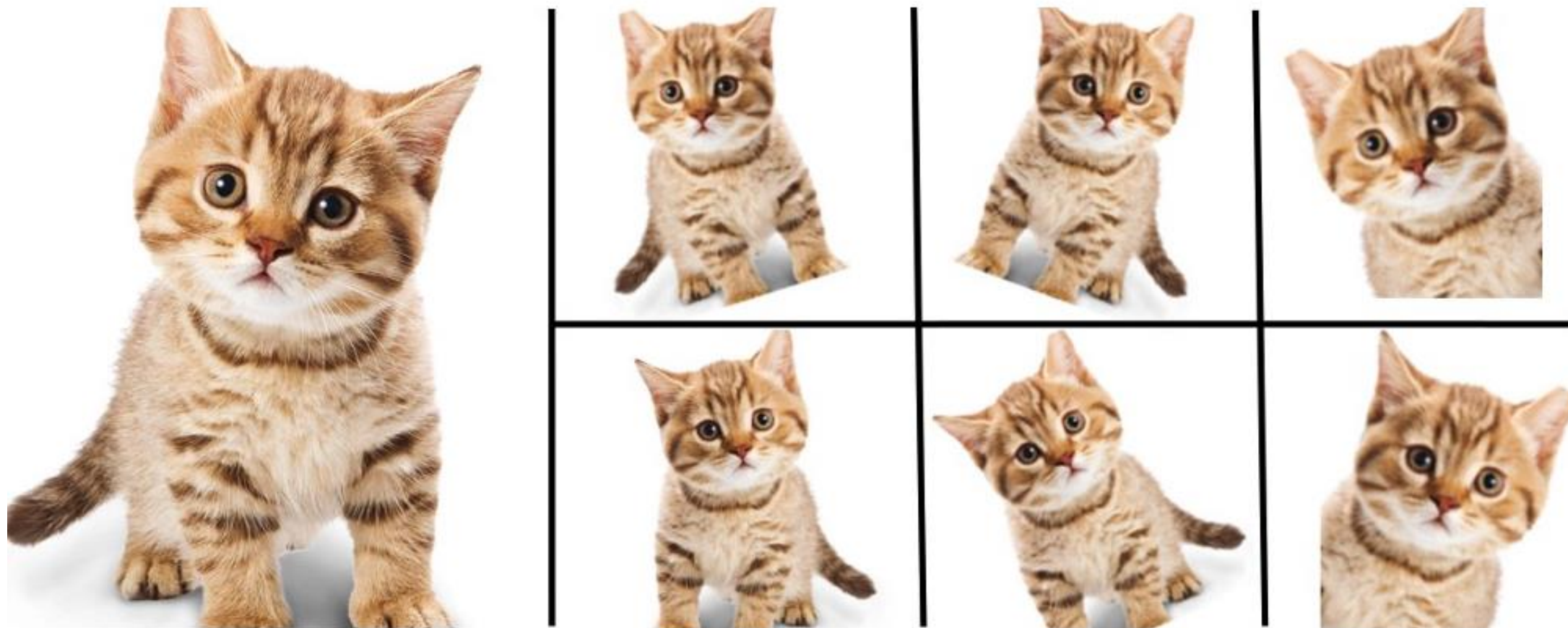
изображения

- симметрии (flip)
- вырезки (crop)
- изменение масштаба (resizing)
- случайные модификации (+шум)
- повороты (rotation)
- сдвиги (shift)
- изменение яркости, контраста, палитры
- эффекты линзы
- синтез / перерисовка изображений (ex GAN)

Тонкость:

преобразования могут переводить объект в другой класс, например повороты «6» и «9»

Расширение обучающего множества (Data Augmentation)



- простая (естественные изменения объектов, ех: небольшие повороты изображений)
- агрессивная («порча» объектов, ех: накладывание масок, объектов других классов)
- креативная (симуляция, GANы и т.п.)

<https://github.com/aleju/imgaug>

<https://medium.com/nanonets/how-to-use-deep-learning-when-you-have-limited-data-part-2-data-augmentation-c26971dc8ced>

Расширение обучающего множества (Data Augmentation): код

```
import torch
import torchvision

transforms = torchvision.transforms.Compose([
    torchvision.transforms.Resize((224,224)),
    torchvision.transforms.ColorJitter(hue=.05, saturation=.05),
    torchvision.transforms.RandomHorizontalFlip(),
    torchvision.transforms.RandomRotation(20, resample=PIL.Image.BILINEAR)
])

dataset = torchvision.datasets.ImageFolder('/data/', transform=transforms)
```

может быть online- и offline- аугментации
м.б. test time- аугментация (TTA)

<https://colab.research.google.com/drive/109vu3F1LTzD1gdVV6cho9fKGx7IzbFI#scrollTo=wpWSjR-HBVso>

Аугментация: Mixup

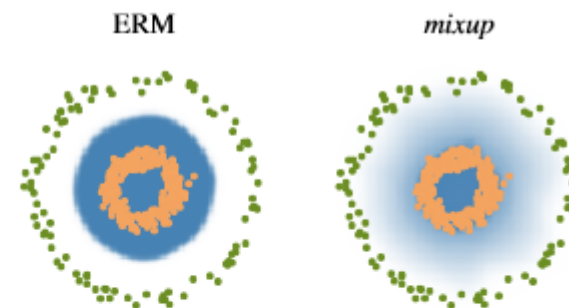
Contribution Motivated by these issues, we introduce a simple and data-agnostic data augmentation routine, termed *mixup* (Section 2). In a nutshell, *mixup* constructs virtual training examples

$$\begin{aligned}\tilde{x} &= \lambda x_i + (1 - \lambda)x_j, & \text{where } x_i, x_j \text{ are raw input vectors} \\ \tilde{y} &= \lambda y_i + (1 - \lambda)y_j, & \text{where } y_i, y_j \text{ are one-hot label encodings}\end{aligned}$$

(x_i, y_i) and (x_j, y_j) are two examples drawn at random from our training data, and $\lambda \in [0, 1]$.

```
# y1, y2 should be one-hot vectors
for (x1, y1), (x2, y2) in zip(loader1, loader2):
    lam = numpy.random.beta(alpha, alpha)
    x = Variable(lam * x1 + (1. - lam) * x2)
    y = Variable(lam * y1 + (1. - lam) * y2)
    optimizer.zero_grad()
    loss(net(x), y).backward()
    optimizer.step()
```

(a) One epoch of *mixup* training in PyTorch.



(b) Effect of *mixup* ($\alpha = 1$) on a toy problem. Green: Class 0. Orange: Class 1. Blue shading indicates $p(y = 1|x)$.

Figure 1: Illustration of *mixup*, which converges to ERM as $\alpha \rightarrow 0$.

<https://github.com/facebookresearch/mixup-cifar10>
<https://arxiv.org/abs/1710.09412>

Аугментация: Mixup

Model	Method	Epochs	Top-1 Error	Top-5 Error
ResNet-50	ERM (Goyal et al., 2017)	90	23.5	-
	<i>mixup</i> $\alpha = 0.2$	90	23.3	6.6
ResNet-101	ERM (Goyal et al., 2017)	90	22.1	-
	<i>mixup</i> $\alpha = 0.2$	90	21.5	5.6
ResNeXt-101 32*4d	ERM (Xie et al., 2016)	100	21.2	-
	ERM	90	21.2	5.6
	<i>mixup</i> $\alpha = 0.4$	90	20.7	5.3
ResNeXt-101 64*4d	ERM (Xie et al., 2016)	100	20.4	5.3
	<i>mixup</i> $\alpha = 0.4$	90	19.8	4.9
ResNet-50	ERM	200	23.6	7.0
	<i>mixup</i> $\alpha = 0.2$	200	22.1	6.1
ResNet-101	ERM	200	22.0	6.1
	<i>mixup</i> $\alpha = 0.2$	200	20.8	5.4
ResNeXt-101 32*4d	ERM	200	21.3	5.9
	<i>mixup</i> $\alpha = 0.4$	200	20.1	5.0

Table 1: Validation errors for ERM and *mixup* on the development set of ImageNet-2012.

SOTA:

CIFAR-10

CIFAR-100

ImageNet-2012

- learning from corrupt labels
- facing adversarial examples
- generalization on speech / tabular data
- to stabilize the training of GANs

Аугментация: Cutout, CutMix

	ResNet-50	Mixup	Cutout	CutMix
Image				
Label	Dog 1.0	Dog 0.5 Cat 0.5	Dog 1.0	Dog 0.6 Cat 0.4
ImageNet Cls (%)	76.3 (+0.0)	77.4 (+1.1)	77.1 (+0.8)	78.4 (+2.1)
ImageNet Loc (%)	46.3 (+0.0)	45.8 (-0.5)	46.7 (+0.4)	47.3 (+1.0)
Pascal VOC Det (mAP)	75.6 (+0.0)	73.9 (-1.7)	75.1 (-0.5)	76.7 (+1.1)

Sangdoo Yun et al «CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features» // <https://arxiv.org/pdf/1905.04899.pdf>

Аугментация: Cutout, CutMix

Let $x \in \mathbb{R}^{W \times H \times C}$ and y denote a training image and its label, respectively. The goal of CutMix is to generate a new training sample $(\tilde{x}, \tilde{y})$ by combining two training samples (x_A, y_A) and (x_B, y_B) . The generated training sample $(\tilde{x}, \tilde{y})$ is used to train the model with its original loss function. We define the combining operation as

$$\begin{aligned}\tilde{x} &= \mathbf{M} \odot x_A + (1 - \mathbf{M}) \odot x_B \\ \tilde{y} &= \lambda y_A + (1 - \lambda) y_B,\end{aligned}\tag{1}$$

where $\mathbf{M} \in \{0, 1\}^{W \times H}$ denotes a binary mask indicating where to drop out and fill in from two images, $\mathbf{1}$ is a binary mask filled with ones, and $\odot$ is element-wise multiplication. Like Mixup [48], the combination ratio λ between two data points is sampled from the beta distribution $\text{Beta}(\alpha, \alpha)$. In our all experiments, we set α to 1, that is λ is sampled from the uniform distribution $(0, 1)$. Note that the major difference is that CutMix replaces an image region with a patch from another training image and generates more locally natural image than Mixup does.

```
lam = np.random.beta(args.beta, args.beta)
rand_index = torch.randperm(input.size()[0]).cuda()
target_a = target
target_b = target[rand_index]
bbx1, bby1, bbx2, bby2 = rand_bbox(input.size(), lam)
input[:, :, bbx1:bbx2, bby1:bby2] = input[rand_index, :,
                                           bbx1:bbx2, bby1:bby2]

# adjust lambda to exactly match pixel ratio
lam = 1 - ((bbx2 - bbx1) * (bby2 - bby1) /
           (input.size()[-1] * input.size()[-2]))

output = model(input)
loss = criterion(output, target_a) * lam +
      criterion(output, target_b) * (1. - lam)
```

есть несоответствие между кодом и статьёй

Аугментация: Attentive CutMix

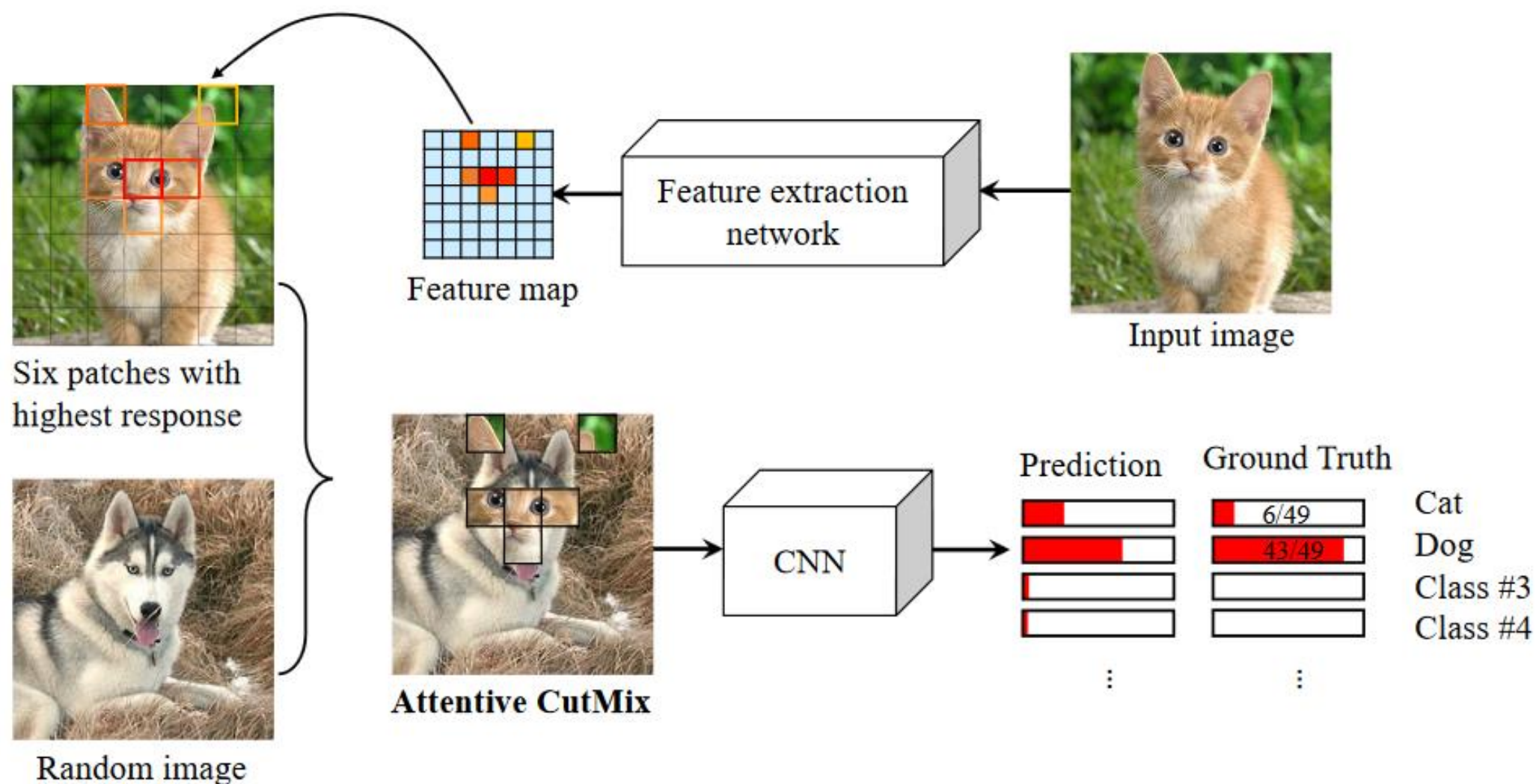


Figure 2: Framework overview of proposed *Attentive CutMix*.

Devesh Walawalkar, Zhiqiang Shen, Zechun Liu, Marios Savvides «Attentive CutMix: An Enhanced Data Augmentation Approach for Deep Learning Based Image Classification» // <https://arxiv.org/pdf/2003.13048.pdf>

Аугментация: GridMask

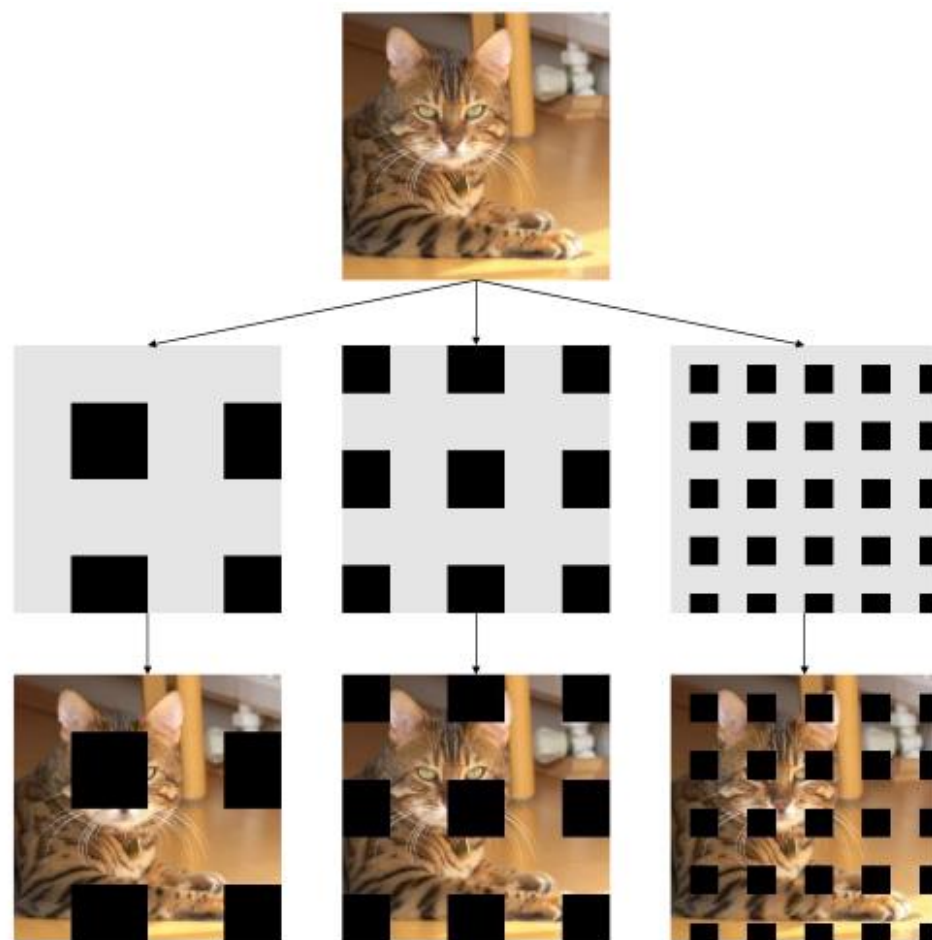


Figure 3. This image shows examples of GridMask. First, we produce a mask according to the given parameters $(r, d, \delta_x, \delta_y)$. Then we multiply it with the input image. The result is shown in the last row. In the mask, gray value is 1, representing the reserved regions; black value is 0, for regions to be deleted.

Model	Accuracy (%)
ResNet18 [10]	95.28
+ Randomerasing [29]	95.32
+ Cutout [4]	96.25
+ HaS [17]	96.10
+ AutoAugment [3]	96.07
+ GridMask (Ours)	96.54
+ AutoAugment & Cutout [3]	96.51
+ AutoAugment & GridMask (Ours)	96.64
WideResNet-28-10 [24]	96.13
+ Radnomerasing [29]*	96.92
+ Cutout [4]	97.04
+ HaS [17]	96.94
+ AutoAugment [3]	97.01
+ GridMask (Ours)	97.24
+ AutoAugment & Cutout [3]	97.39
+ AutoAugment & GridMask (Ours)	97.48
ShakeShake-26-32 [5]	96.42
+ Randomerasing [29]	96.46
+ Cutout [4]	96.96
+ Has [17]	96.89
+ Autoaugment [3]	96.96
+ GridMask (Ours)	97.20
+ AutoAugment & Cutout [3]	97.36
+ AutoAugment & GridMask (Ours)	97.42

Table 3. Results on CIFAR10 are summarized in this table. We achieve the best accuracy on different models. * means results reported in the original paper.

P Chen «GridMask Data Augmentation» <https://arxiv.org/pdf/2001.04086.pdf>

Ансамбль нейросетей

- псевдо-ансамбль (pseudo-ensemble) (общая свёрточная часть, разные FC-блоки)
- несколько независимых моделей, усредняем результат (как всегда +2%)
- несколько независимых моделей, усредняем веса (не работает п.в.)
- Snapshot Ensemble – усреднение на разных эпохах обучения (аналог усреднения Поляка)
- усреднение параметров на одной траектории оптимизации – Stochastic Weight Averaging (SWA)

+ более сложные ансамбли...

Model	Prediction method	Test Accuracy
Baseline (10 epochs)	Single model	0.837
True ensemble of 10 models	Average predictions	0.855
True ensemble of 10 models	Voting	0.851
Snapshots (25) over 10 epochs	Average predictions	0.865
Snapshots (25) over 10 epochs	Voting	0.861
Snapshots (25) over 10 epochs	Parameter averaging	0.864

Ансамбль нейросетей: Snapshot Ensembles

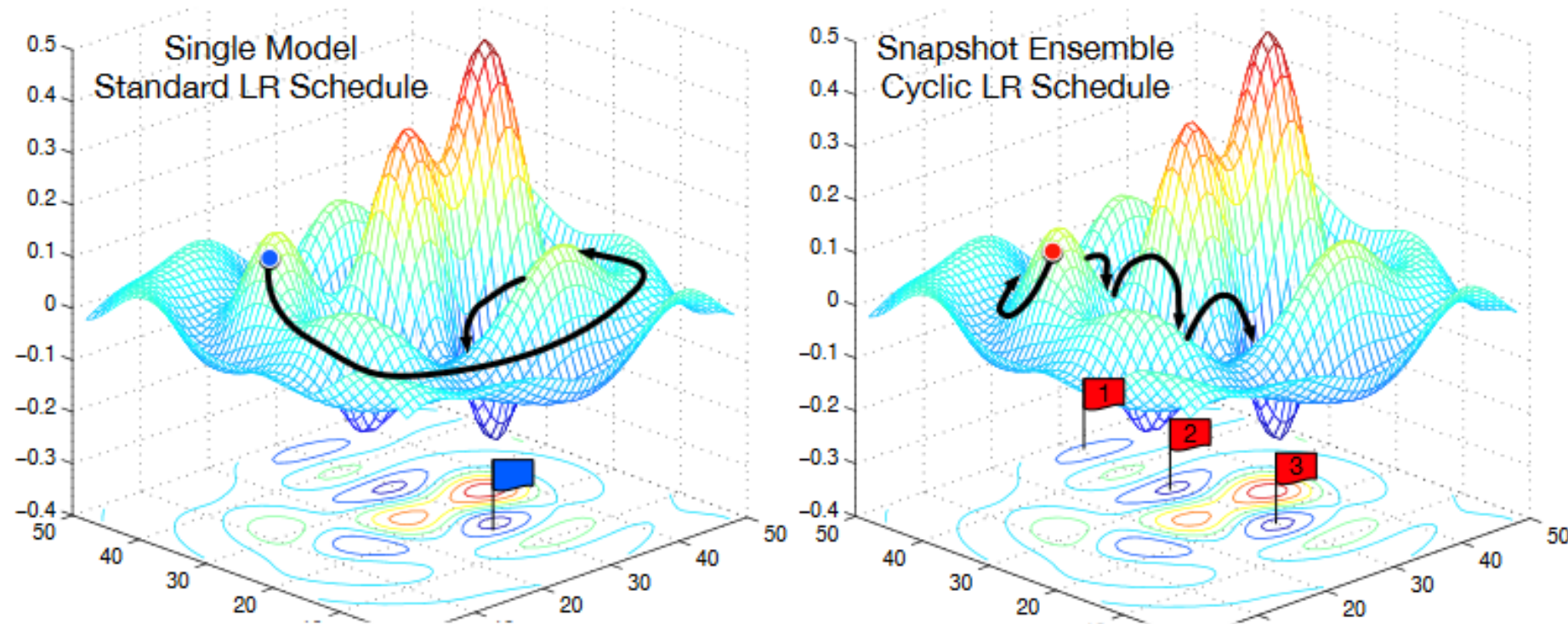


Figure 1: **Left:** Illustration of SGD optimization with a typical learning rate schedule. The model converges to a minimum at the end of training. **Right:** Illustration of Snapshot Ensembling. The model undergoes several learning rate annealing cycles, converging to and escaping from multiple local minima. We take a snapshot at each minimum for test-time ensembling.

Gao Huang, Yixuan Li, Geoff Pleiss, Zhuang Liu, John E. Hopcroft, Kilian Q. Weinberger «Snapshot Ensembles: Train 1, get M for free» // <https://arxiv.org/pdf/1704.00109.pdf>

Ансамбль нейросетей: Stochastic Weight Averaging (SWA)

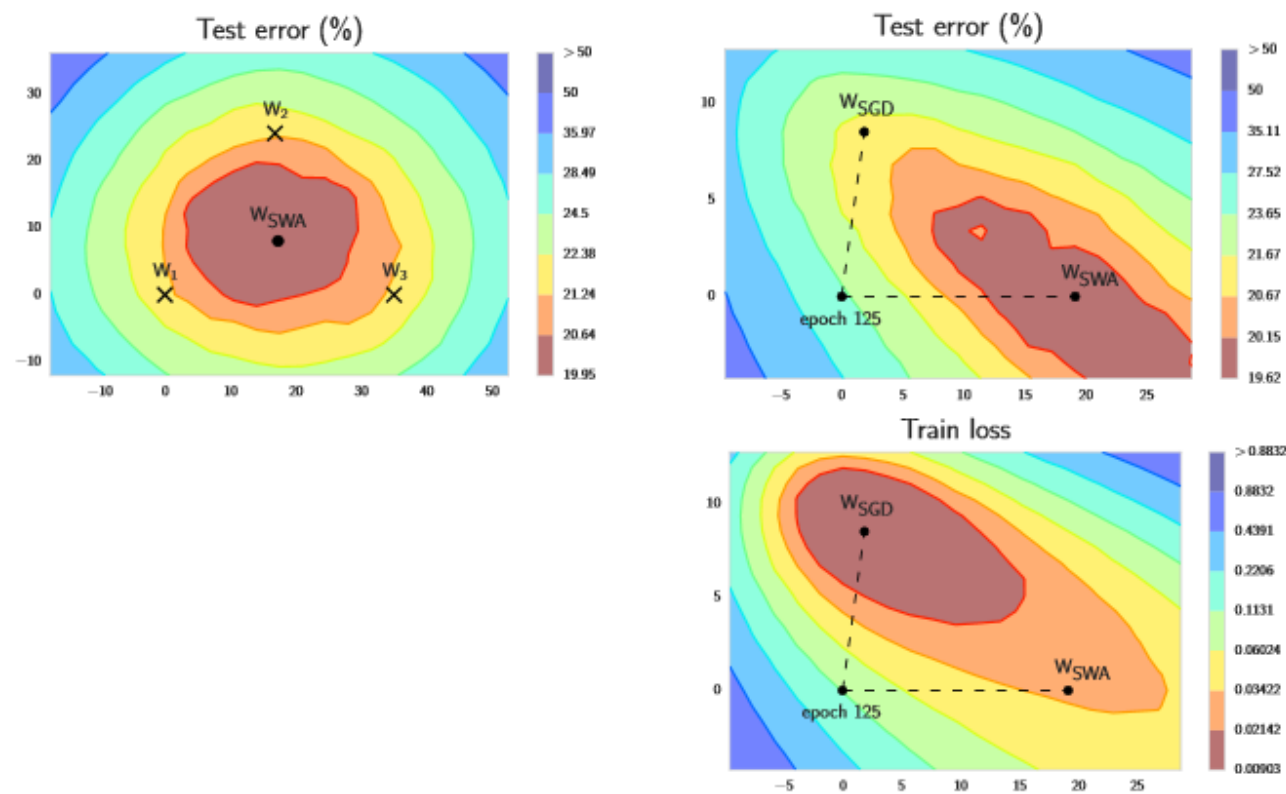


Figure 1: Illustrations of SWA and SGD with a Preactivation ResNet-164 on CIFAR-100¹. **Left:** test error surface for three FGE samples and the corresponding SWA solution (averaging in weight space). **Middle and Right:** test error and train loss surfaces showing the weights proposed by SGD (at convergence) and SWA, starting from the same initialization of SGD after 125 training epochs.

Pavel Izmailov et. al. «Averaging Weights Leads to Wider Optima and Better Generalization» // <https://arxiv.org/abs/1803.05407>

Algorithm 1 Stochastic Weight Averaging

Require:

weights $\hat{w}$, LR bounds α_1, α_2 ,
cycle length c (for constant learning rate $c = 1$), number of iterations n

Ensure: w_{SWA}

$w \leftarrow \hat{w}$ {Initialize weights with $\hat{w}$ }

$w_{\text{SWA}} \leftarrow w$

for $i \leftarrow 1, 2, \dots, n$ **do**

$\alpha \leftarrow \alpha(i)$ {Calculate LR for the iteration}

$w \leftarrow w - \alpha \nabla \mathcal{L}_i(w)$ {Stochastic gradient update}

if $\text{mod}(i, c) = 0$ **then**

$n_{\text{models}} \leftarrow i/c$ {Number of models}

$w_{\text{SWA}} \leftarrow \frac{w_{\text{SWA}} \cdot n_{\text{models}} + w}{n_{\text{models}} + 1}$ {Update average}

end if

end for

{Compute BatchNorm statistics for w_{SWA} weights}

Ансамбль нейросетей: Stochastic Weight Averaging (SWA)

Table 2: Top-1 accuracies (%) on ImageNet for SWA and SGD with different architectures.

DNN	SGD	SWA	
		5 epochs	10 epochs
ResNet-50	76.15	76.83 ± 0.01	76.97 ± 0.05
ResNet-152	78.31	78.82 ± 0.01	78.94 ± 0.07
DenseNet-161	77.65	78.26 ± 0.09	78.44 ± 0.06

For all 3 architectures SWA provides consistent improvement by 0.6-0.9% over the pretrained models.

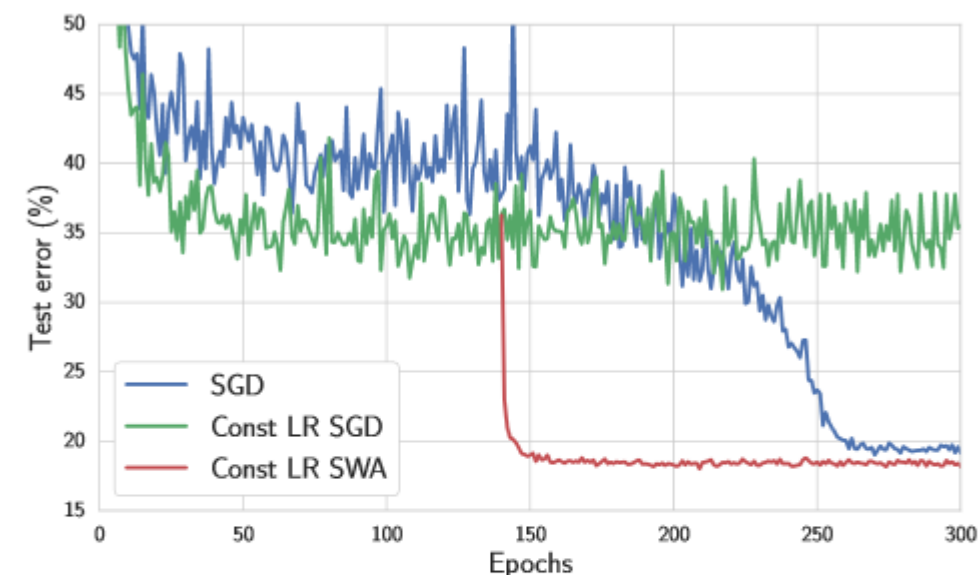


Figure 7: Test error as a function of training epoch for constant (green) and decaying (blue) learning rate schedules for a Wide ResNet-28-10 on CIFAR-100. In red we average the points along the trajectory of SGD with constant learning rate starting at epoch 140.

Ансамбль нейросетей: более сложные

Algorithm 1 Pseudocode of the training procedure for our method

- 1: $\triangleright$ Let each neural network parametrize a distribution over the outputs, i.e. $p_{\theta}(y|\mathbf{x})$. Use a proper scoring rule as the training criterion $\ell(\theta, \mathbf{x}, y)$. Recommended default values are $M = 5$ and $\epsilon = 1\%$ of the input range of the corresponding dimension (e.g 2.55 if input range is $[0, 255]$).
 - 2: Initialize $\theta_1, \theta_2, \dots, \theta_M$ randomly
 - 3: **for** $m = 1 : M$ **do** $\triangleright$ train networks independently in parallel
 - 4: Sample data point n_m randomly for each net $\triangleright$ single n_m for clarity, minibatch in practice
 - 5: Generate adversarial example using $\mathbf{x}'_{n_m} = \mathbf{x}_{n_m} + \epsilon \text{sign}(\nabla_{\mathbf{x}_{n_m}} \ell(\theta_m, \mathbf{x}_{n_m}, y_{n_m}))$
 - 6: Minimize $\ell(\theta_m, \mathbf{x}_{n_m}, y_{n_m}) + \ell(\theta_m, \mathbf{x}'_{n_m}, y_{n_m})$ w.r.t. θ_m $\triangleright$ adversarial training (optional)
-

We treat the ensemble as a uniformly-weighted mixture model and combine the predictions as $p(y|\mathbf{x}) = M^{-1} \sum_{m=1}^M p_{\theta_m}(y|\mathbf{x}, \theta_m)$. For classification, this corresponds to averaging the predicted probabilities. For regression, the prediction is a mixture of Gaussian distributions. For ease of computing quantiles and predictive probabilities, we further approximate the ensemble prediction as a Gaussian whose mean and variance are respectively the mean and variance of the mixture. The mean and variance of a mixture $M^{-1} \sum \mathcal{N}(\mu_{\theta_m}(\mathbf{x}), \sigma_{\theta_m}^2(\mathbf{x}))$ are given by $\mu_*(\mathbf{x}) = M^{-1} \sum_m \mu_{\theta_m}(\mathbf{x})$ and $\sigma_*^2(\mathbf{x}) = M^{-1} \sum_m (\sigma_{\theta_m}^2(\mathbf{x}) + \mu_{\theta_m}^2(\mathbf{x})) - \mu_*^2(\mathbf{x})$ respectively.

Использование «сопоставительных примеров»

Balaji Lakshminarayanan «Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles» // <https://arxiv.org/pdf/1612.01474.pdf>

Диагностика проблем с НС

1. Численная проверка
2. Визуализация
3. Маленькая выборка
4. Насыщенность нейронов
5. Динамика ошибки
6. Изменения весов

Диагностика проблем с НС: Численная проверка

1. Численно проверить градиенты (с помощью конечных разностей)

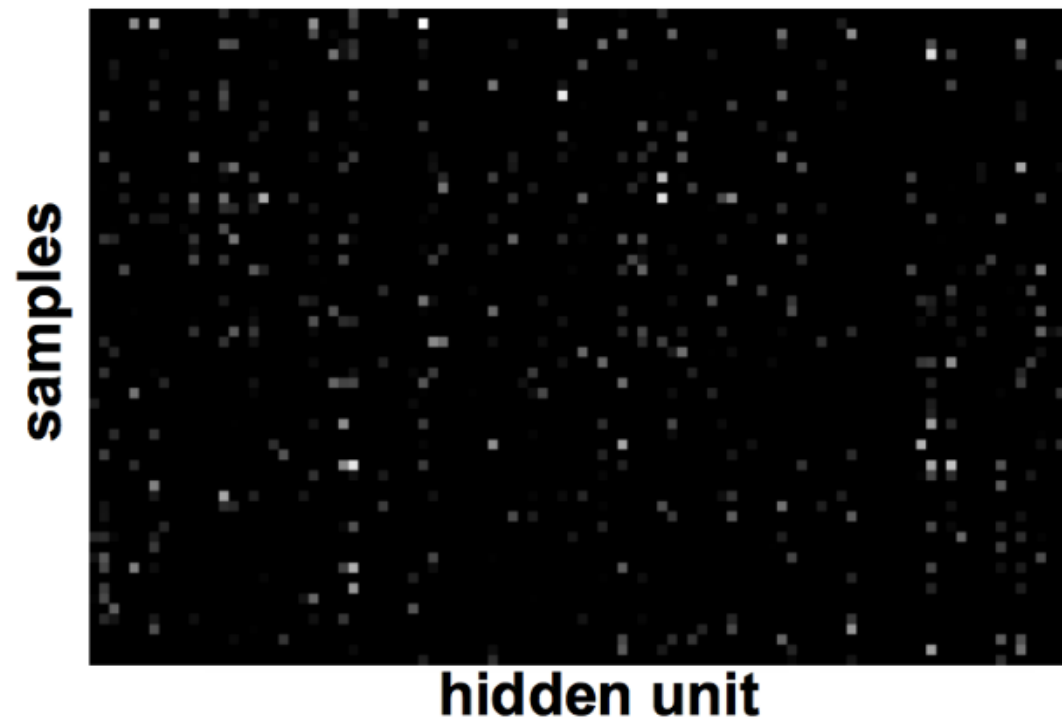
$$\frac{\partial f(x)}{\partial x} \approx \frac{f(x + \varepsilon) - f(x - \varepsilon)}{2\varepsilon}$$

проверка реализации обратного и прямого распространения

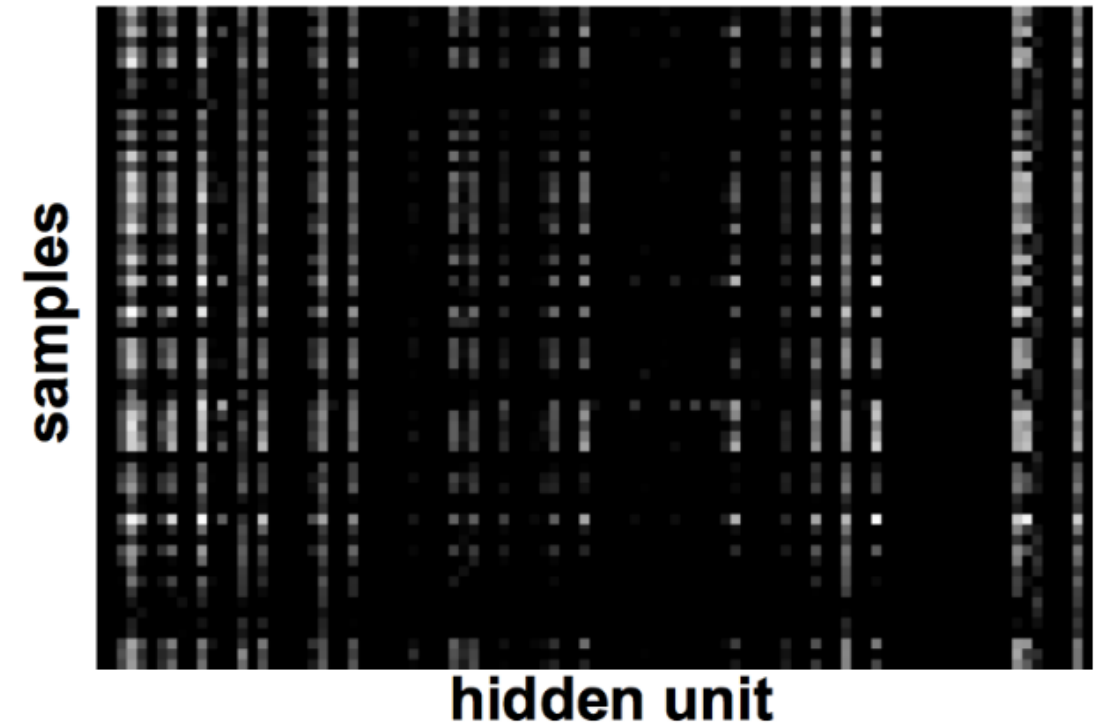
Диагностика проблем с НС: Визуализация

Признаки (в общем смысле) должны быть некоррелированными и с большой дисперсией

Хорошо:



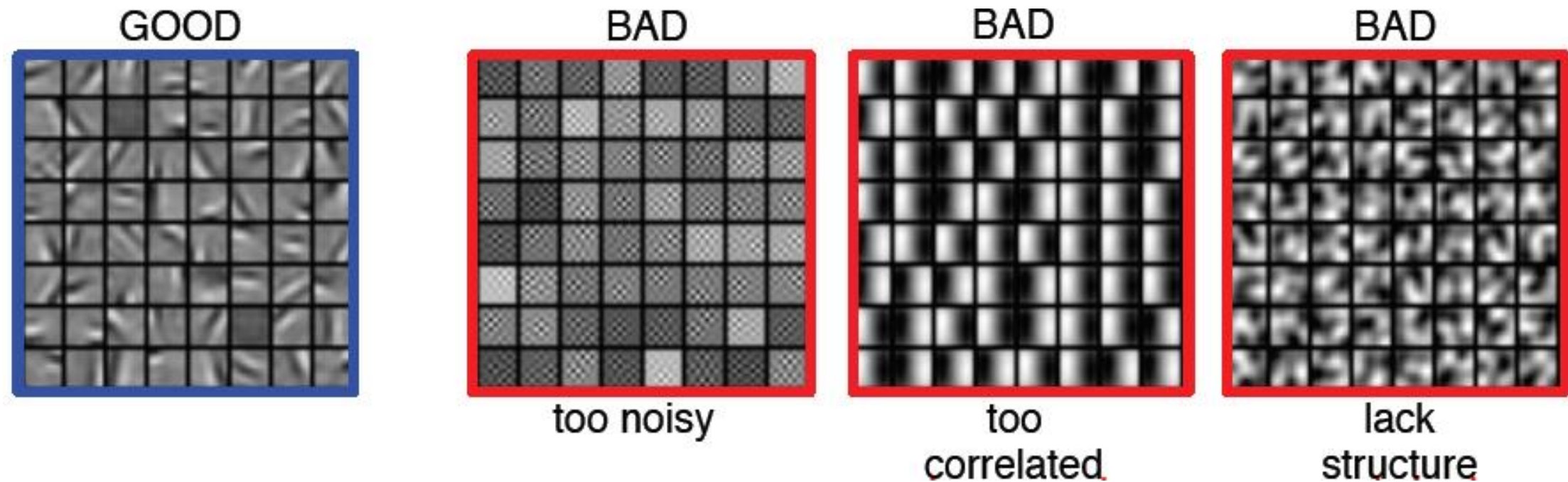
Плохо:



Marc'Aurelio Ranzato, CVPR 2014

Диагностика проблем с НС: Визуализация

Хорошие фильтры имеют структуру и некоррелированные



Диагностика проблем с НС: Маленькая выборка

Убедиться, что сеть работает на небольшом куске данных
(~ 100 – 500 объектов)

Диагностика проблем с НС: Насыщенность нейронов

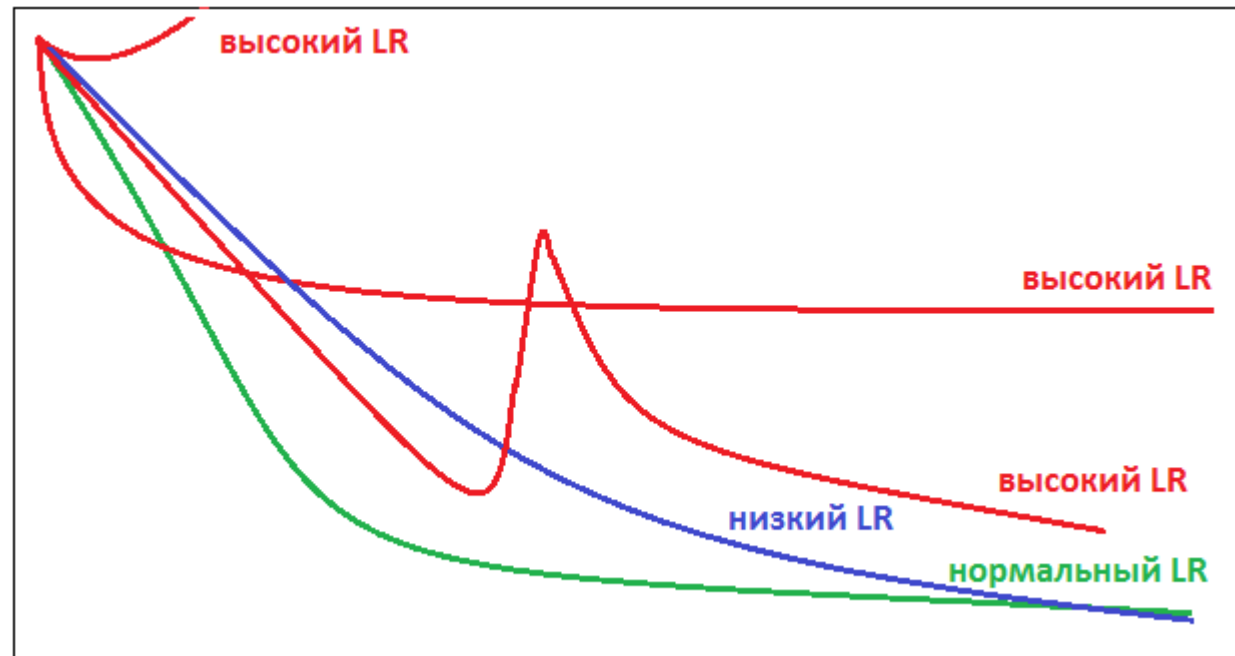
Насыщены ли нейроны ещё до обучения?
Нормировка!

Диагностика проблем с НС: Изменения весов

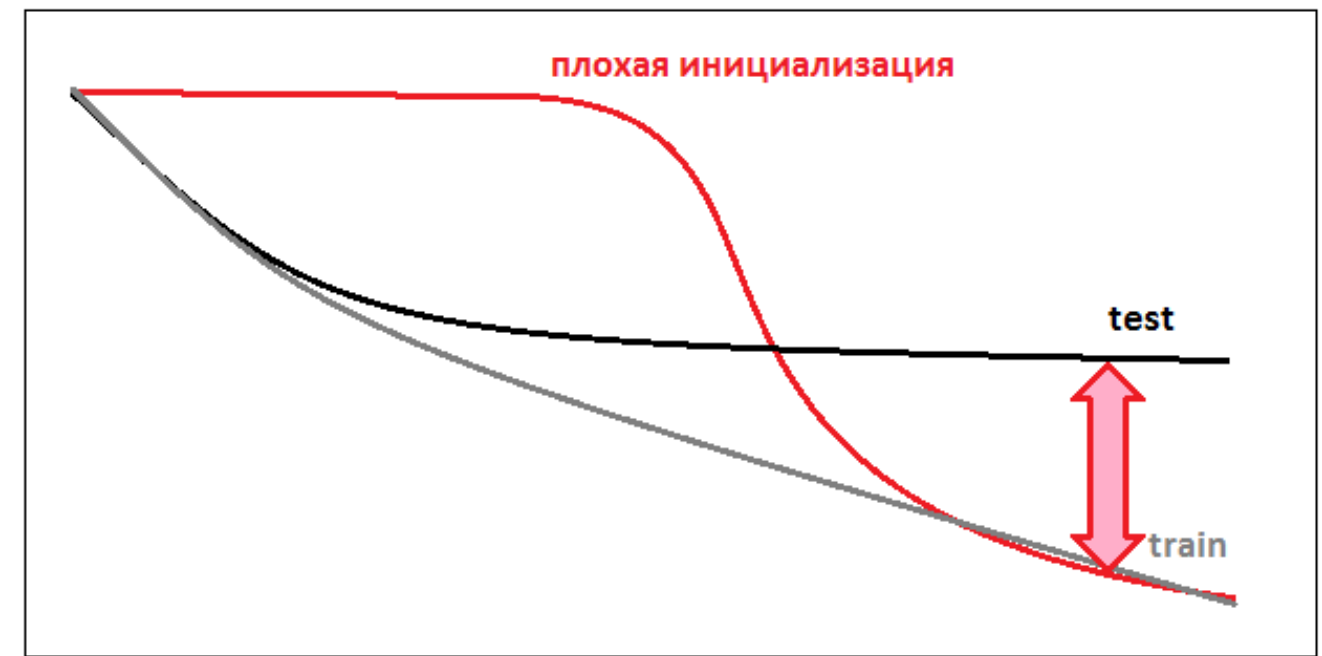
Насколько меняются веса за итерацию (~ 0.1%)

Диагностика проблем с НС: Динамика ошибки

Как ведёт себя ошибка обучения – настроить темп обучения!



настройка темпа



плохая инициализация
м.б. плохая архитектура,
ex: нет прокидывания связей

Большой зазор $\Rightarrow$ переобучение $\Rightarrow$ усилить регуляризацию
Маленький $\Rightarrow$ усложнить модель

Диагностика проблем с НС: Динамика ошибки

На что ещё смотреть при обучении:

- ошибка / точность (метрика задачи) на обучении / валидации
 - нормы градиентов по слоям (распределения)
 - активации (гистограммы активаций)
 - время одной итерации
(ускоряйте обучение: GPU, данные с SSD)
 - прогресс обучения: когда заметны какие-то паттерны
(пример: в синтезе речи должно появиться что-то осмысленное
через несколько часов)

Настройка темпа обучения

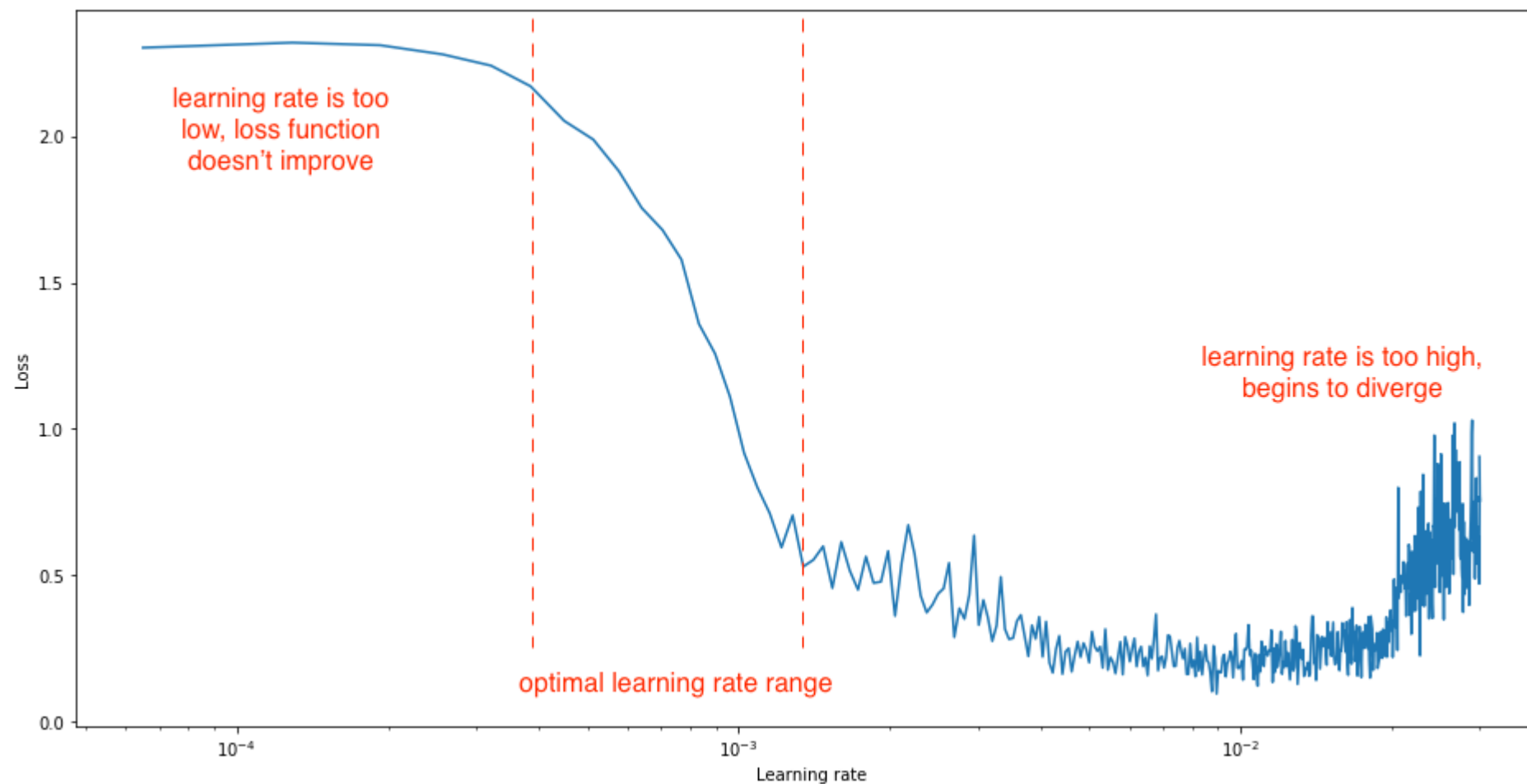
При фиксированном темпе:
чем больше – быстрее идём,
но и сильнее перескакиваем
(будем прыгать вокруг оптимума)

на практике снижают темп!
начинают с такого, что метод не расходится

рекомендации:
глубже – меньше
меньше батч – меньше темп
SGD: 0.1
Adam: 1e-3
Adam + transformer: 1e-5



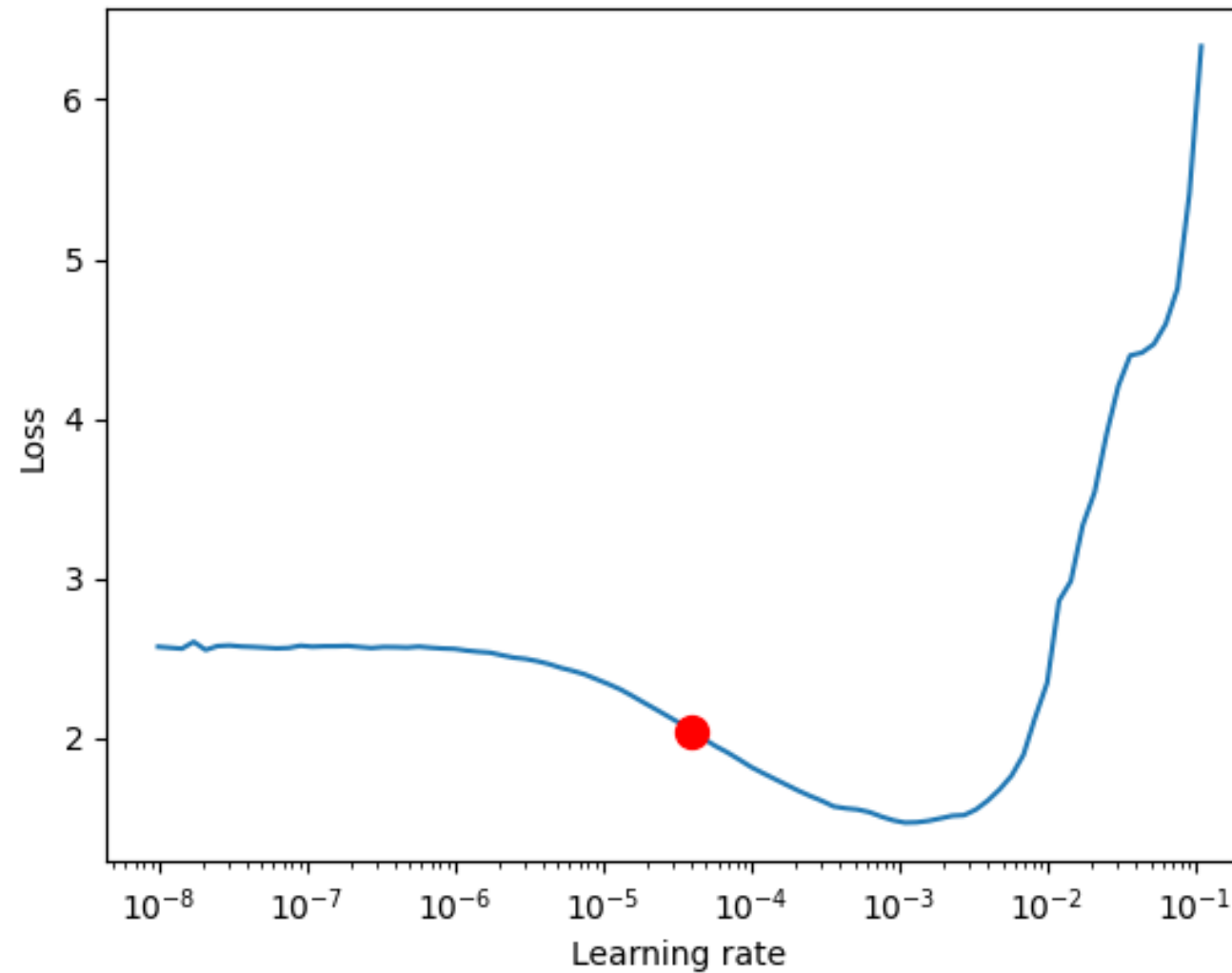
Настройка темпа обучения



<https://www.jeremyjordan.me/nn-learning-rate/>

Аналогичная идея – увеличивать темп после каждого батча, по графику ошибки найти оптимальный темп:
«Cyclical Learning Rates for Training Neural Networks» <https://arxiv.org/abs/1506.01186>

Настройка темпа обучения



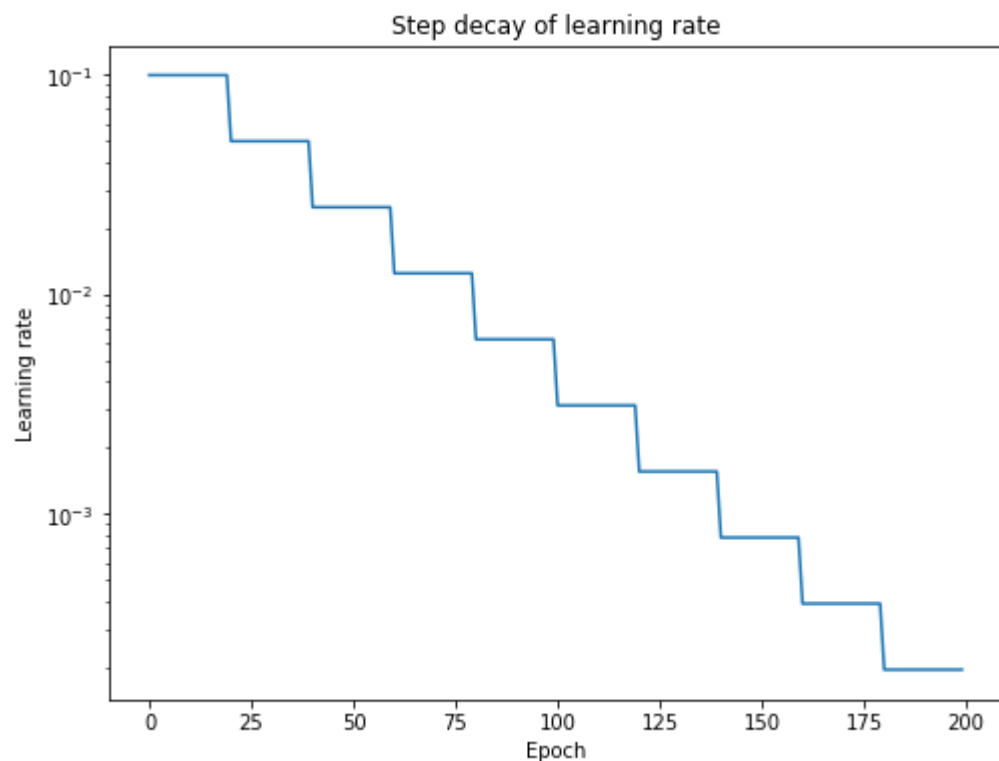
https://pytorch-lightning.readthedocs.io/en/latest/advanced/lr_finder.html

Настройка темпа обучения: Learning Rate Annealing

Всегда смотрите на кривые ошибок во время обучения!

уменьшать через каждые N эпох

```
torch.optim.lr_scheduler.StepLR(optimizer,  
                                step_size=30,  
                                gamma=0.1,  
                                last_epoch=-1,  
                                verbose=False)
```



```
>>> # lr = 0.05      if epoch < 30  
>>> # lr = 0.005     if 30 <= epoch < 60  
>>> # lr = 0.0005    if 60 <= epoch < 90
```

<https://www.jeremyjordan.me/nn-learning-rate/>

Настройка темпа обучения: Learning Rate Schedule

уменьшать после выхода на плато
(качество не улучшается в течение N эпох)
или какого-то другого специфического условия

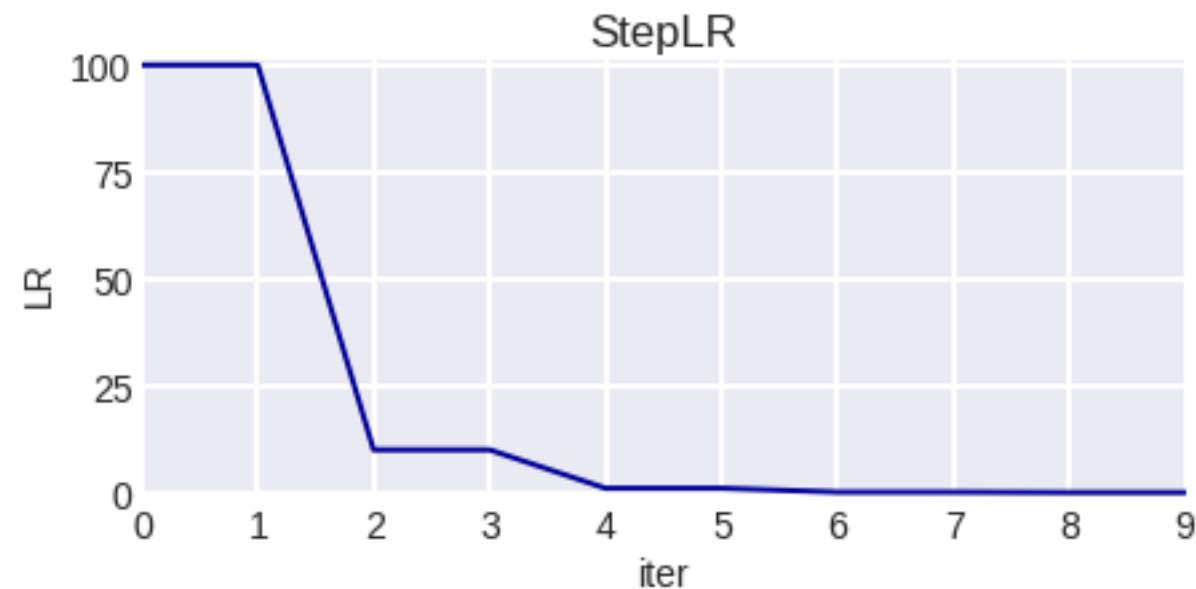
увеличивать при попадании в седловую точку
(отсутствии прогресса)

`torch.optim.lr_scheduler.`

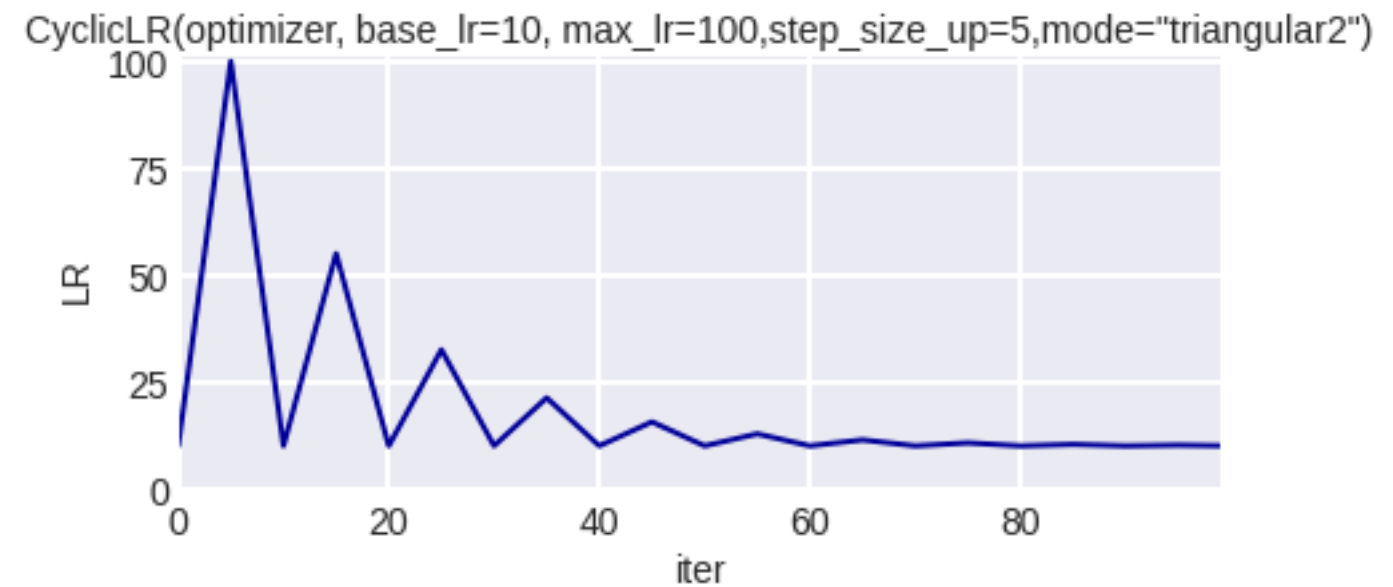
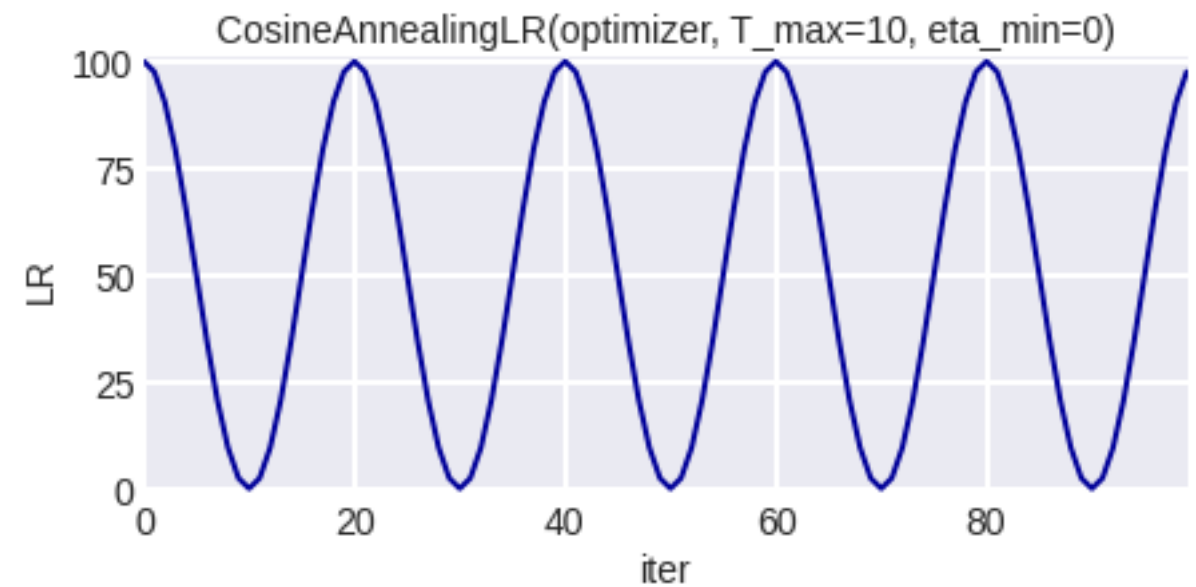
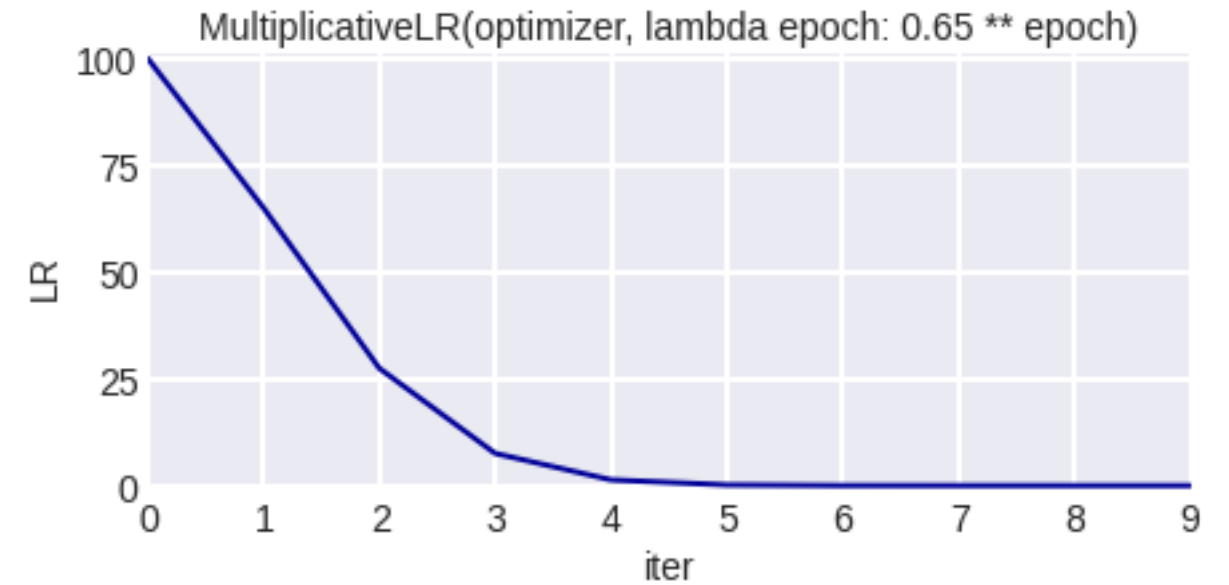
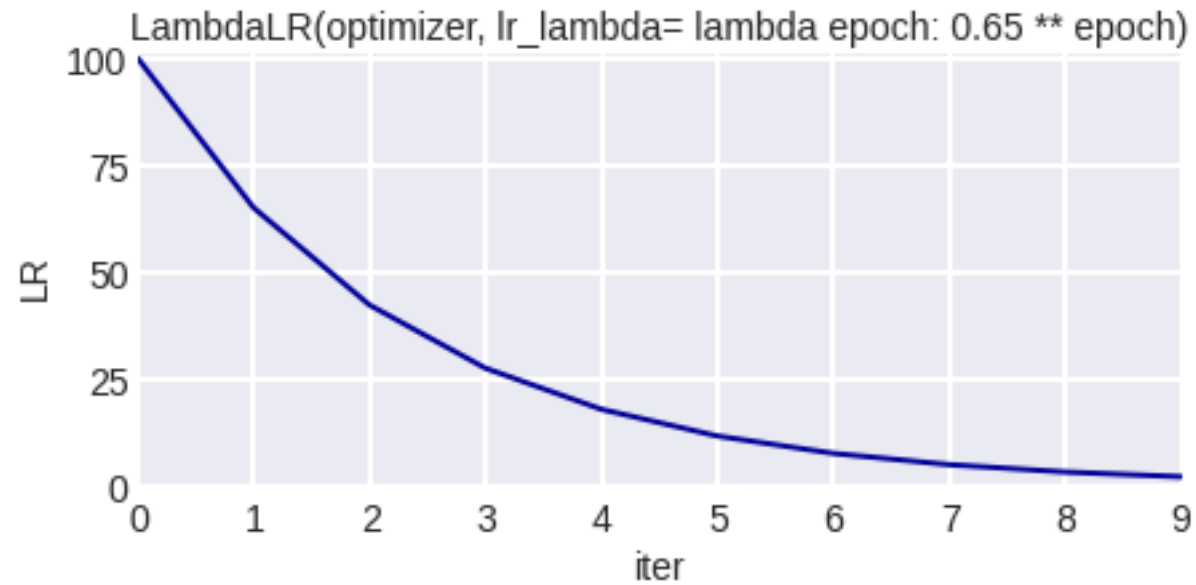
Минутка кода: сценарий изменения темпа

```
model = torch.nn.Linear(2, 1)
optimizer = torch.optim.SGD(model.parameters(), lr=100)
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=2, gamma=0.1)
lrs = []

for i in range(10):
    optimizer.step()
    lrs.append(optimizer.param_groups[0]["lr"])
    scheduler.step()
```



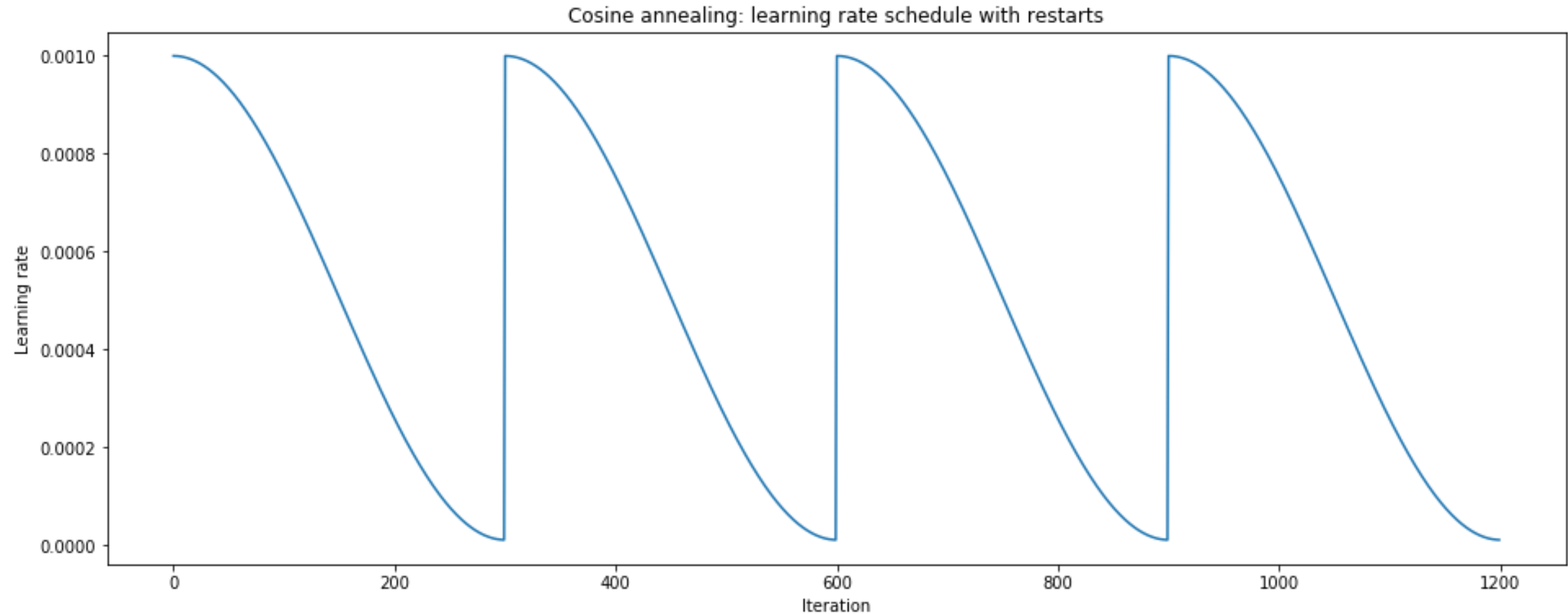
Сценарий изменения темпа



<https://www.kaggle.com/isbhargav/guide-to-pytorch-learning-rate-scheduling>

Настройка темпа обучения

Stochastic Gradient Descent with Warm Restarts (SGDR)



и можно строить ансамбль моделей в точках затухания!

<https://www.jeremyjordan.me/nn-learning-rate/>

Настройка темпа обучения

Optimized learning rate schedule

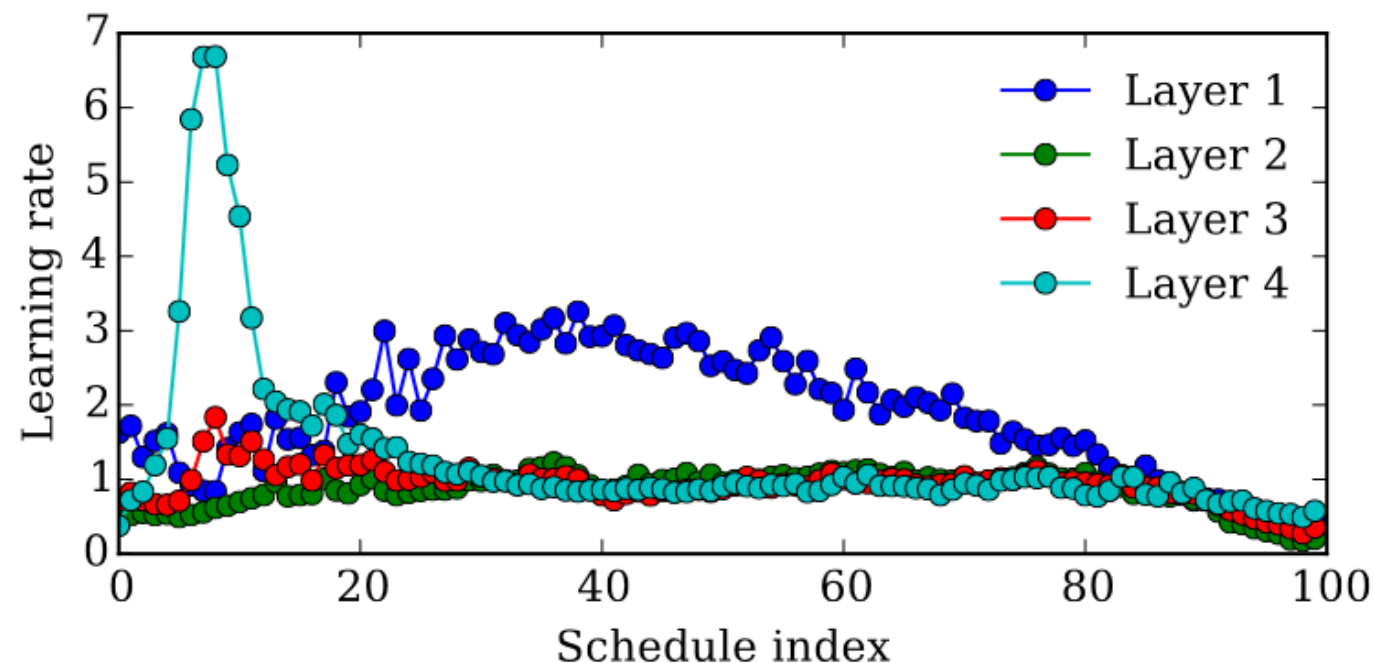


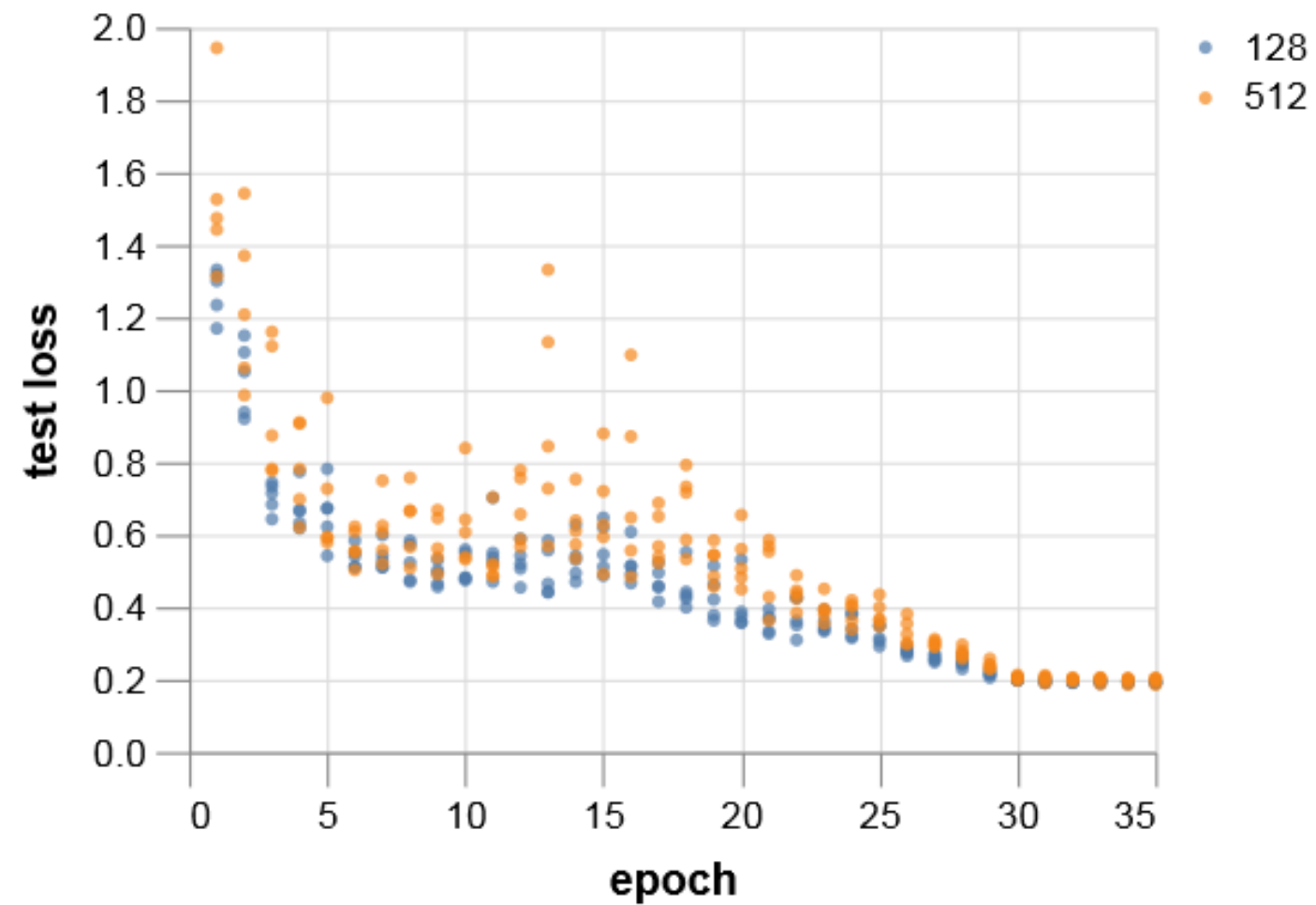
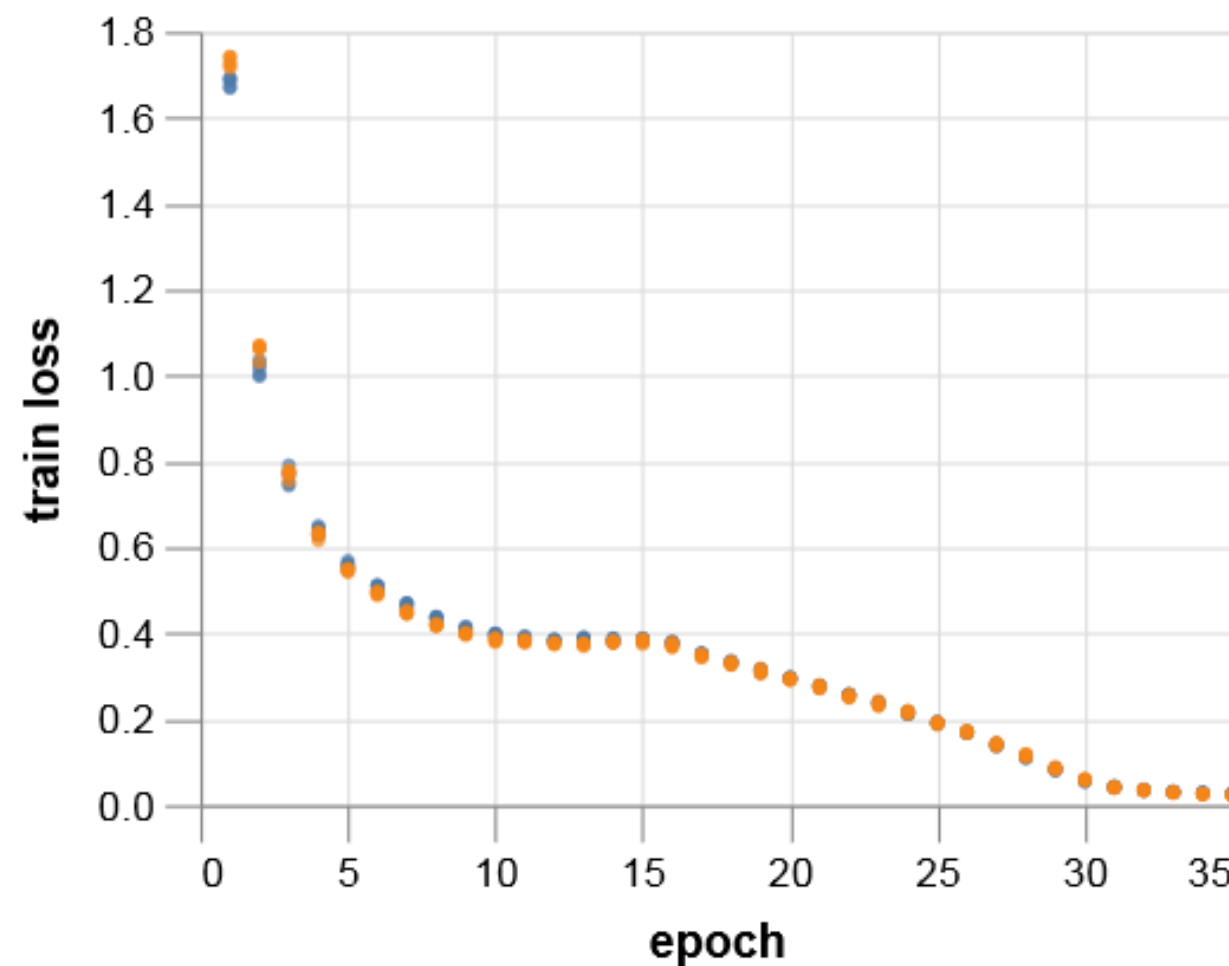
Figure 2. A learning-rate training schedule for the weights in each layer of a neural network, optimized by hypergradient descent. The optimized schedule starts by taking large steps only in the topmost layer, then takes larger steps in the first layer. All layers take smaller step sizes in the last 10 iterations. Not shown are the schedules for the biases or the momentum, which showed less structure.

Dougal Maclaurin, David Duvenaud, Ryan P. Adams «Gradient-based Hyperparameter Optimization through Reversible Learning» // <https://arxiv.org/abs/1502.03492>

Диагностика проблем с НС

Недообучение	Другие методы оптимизации GPU Сложнее сеть
Переобучение	Упрощение сети Регуляризация (+Dropout...) приёмы, которые были Обучение без учителя (более сложная задача $\Rightarrow$ меньше переобучения)

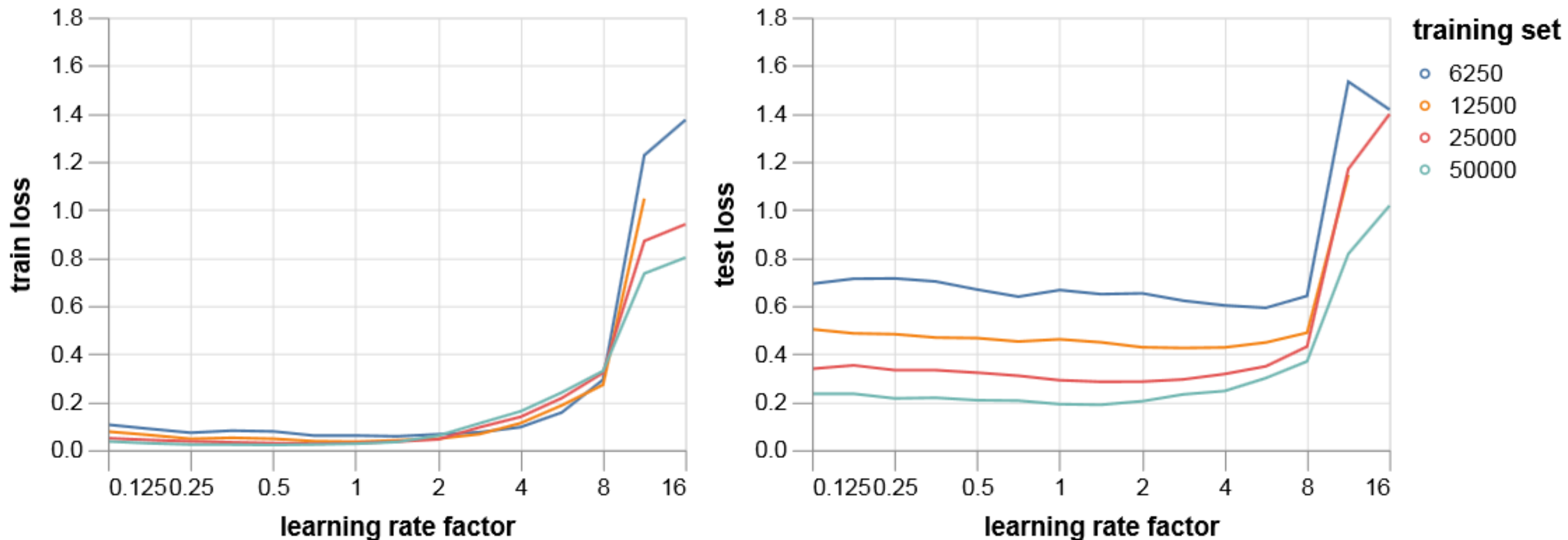
Обучение нейронных сетей



зависимость от размера батча

<https://myrtle.ai/learn/how-to-train-your-resnet-2-mini-batches/>

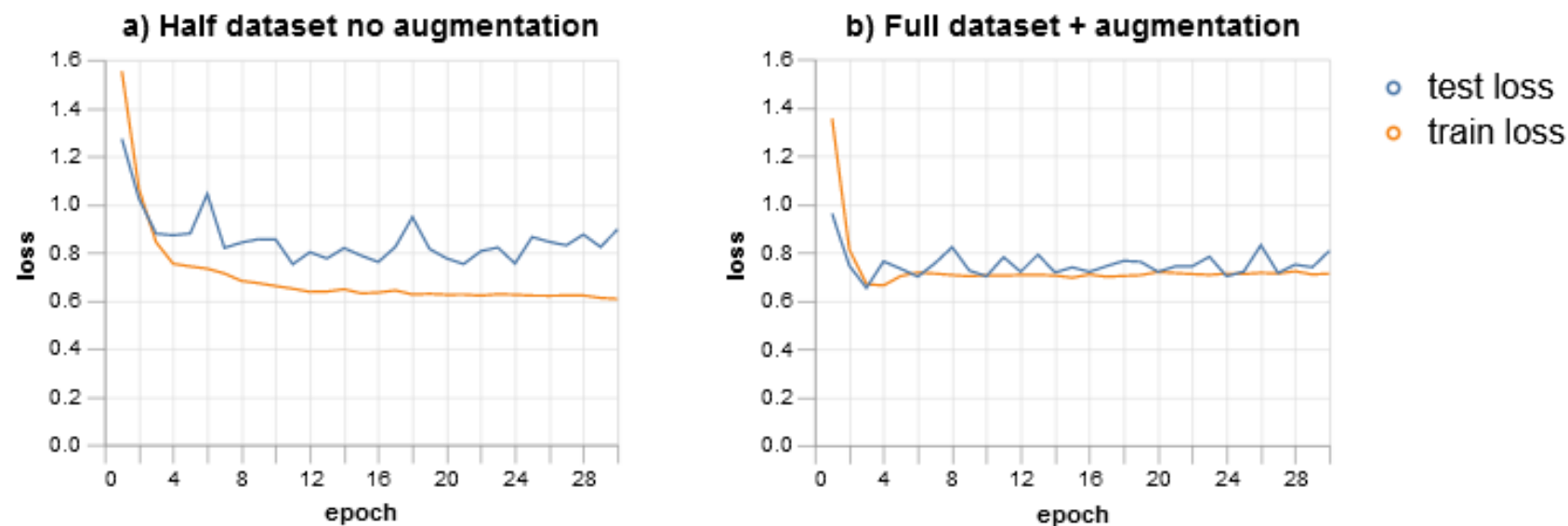
Обучение нейронных сетей



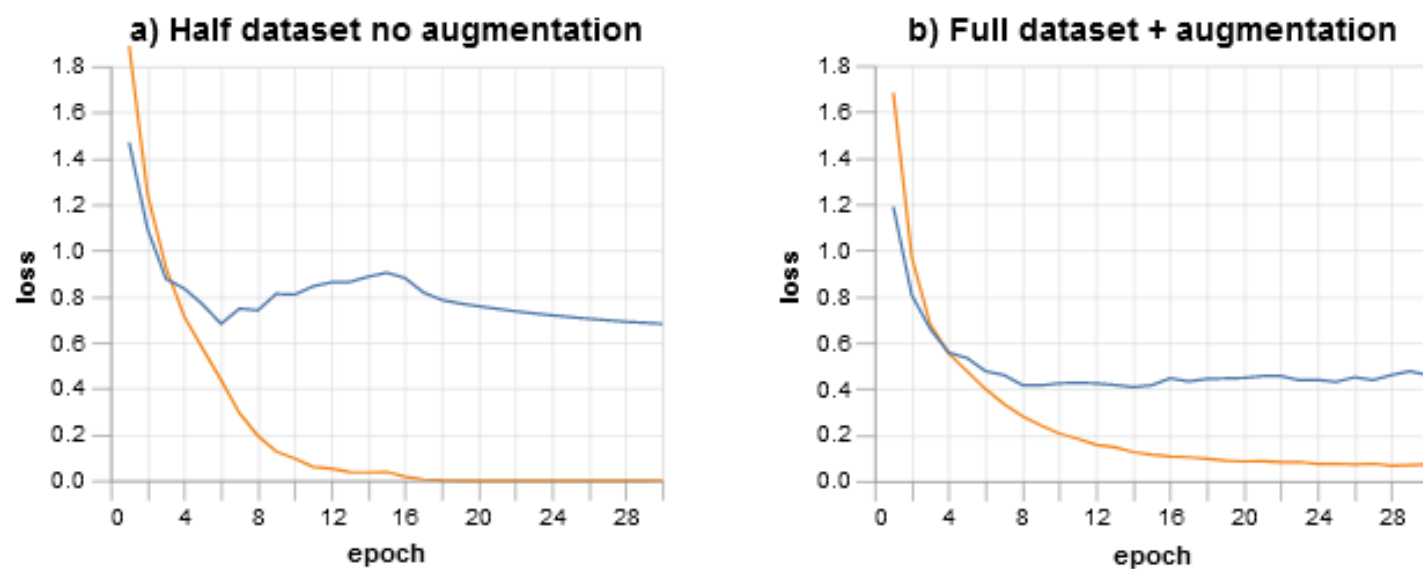
зависимость от темпа обучения при разном размере обучения

Обучение нейронных сетей

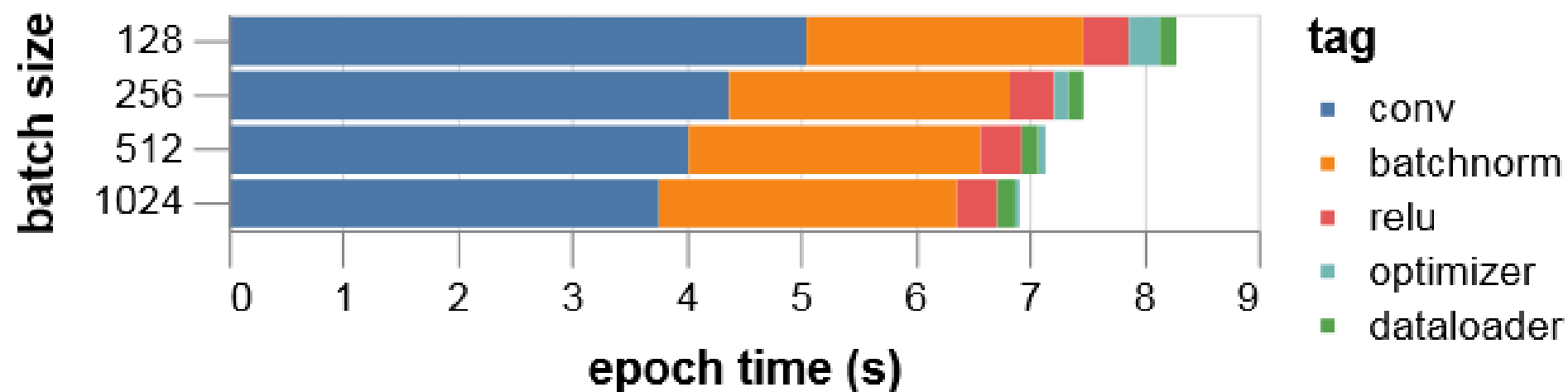
maximum learning rate 4× higher than the original training setup:



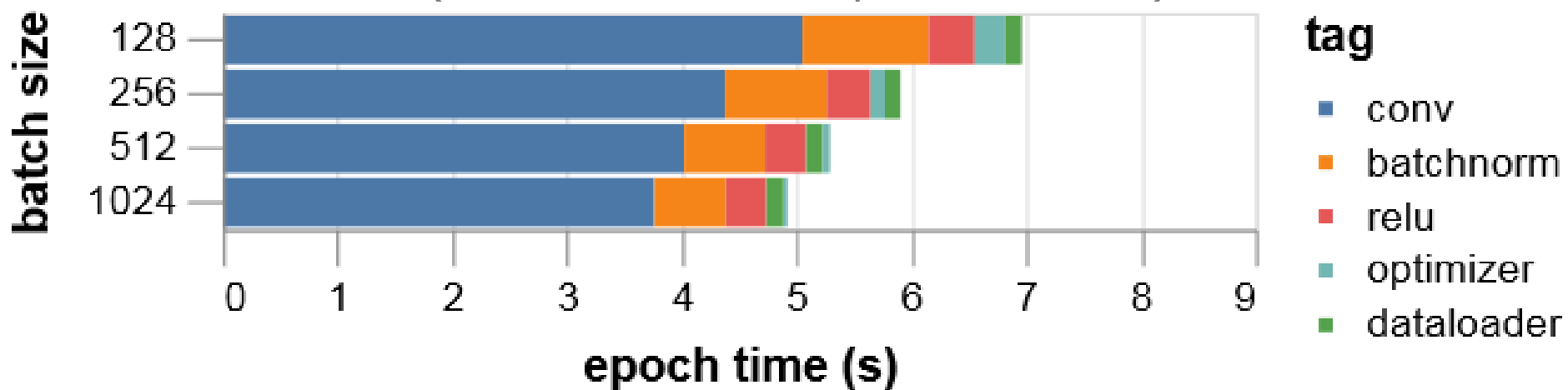
maximum learning rate 4× lower than the original training:



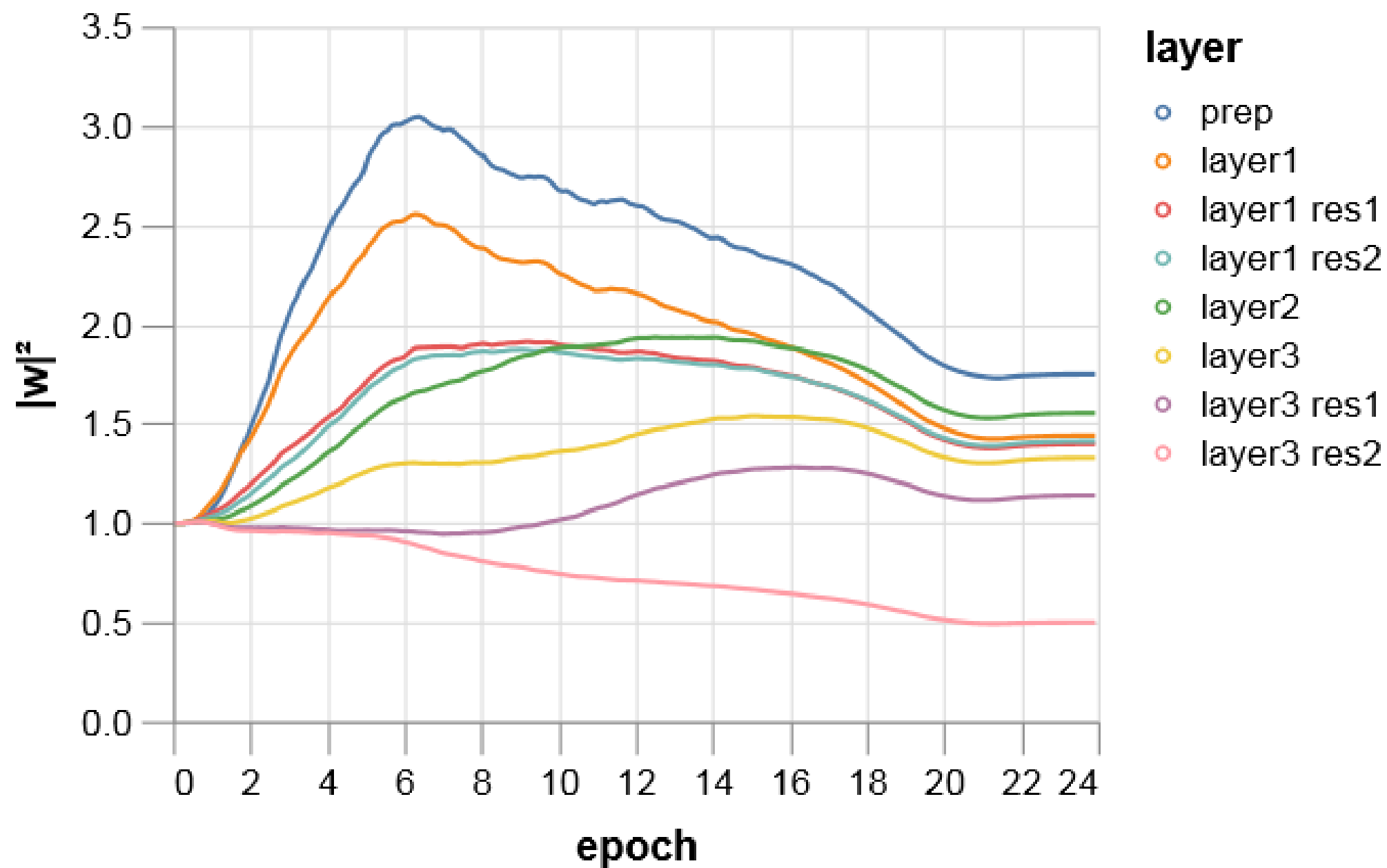
Обучение нейронных сетей



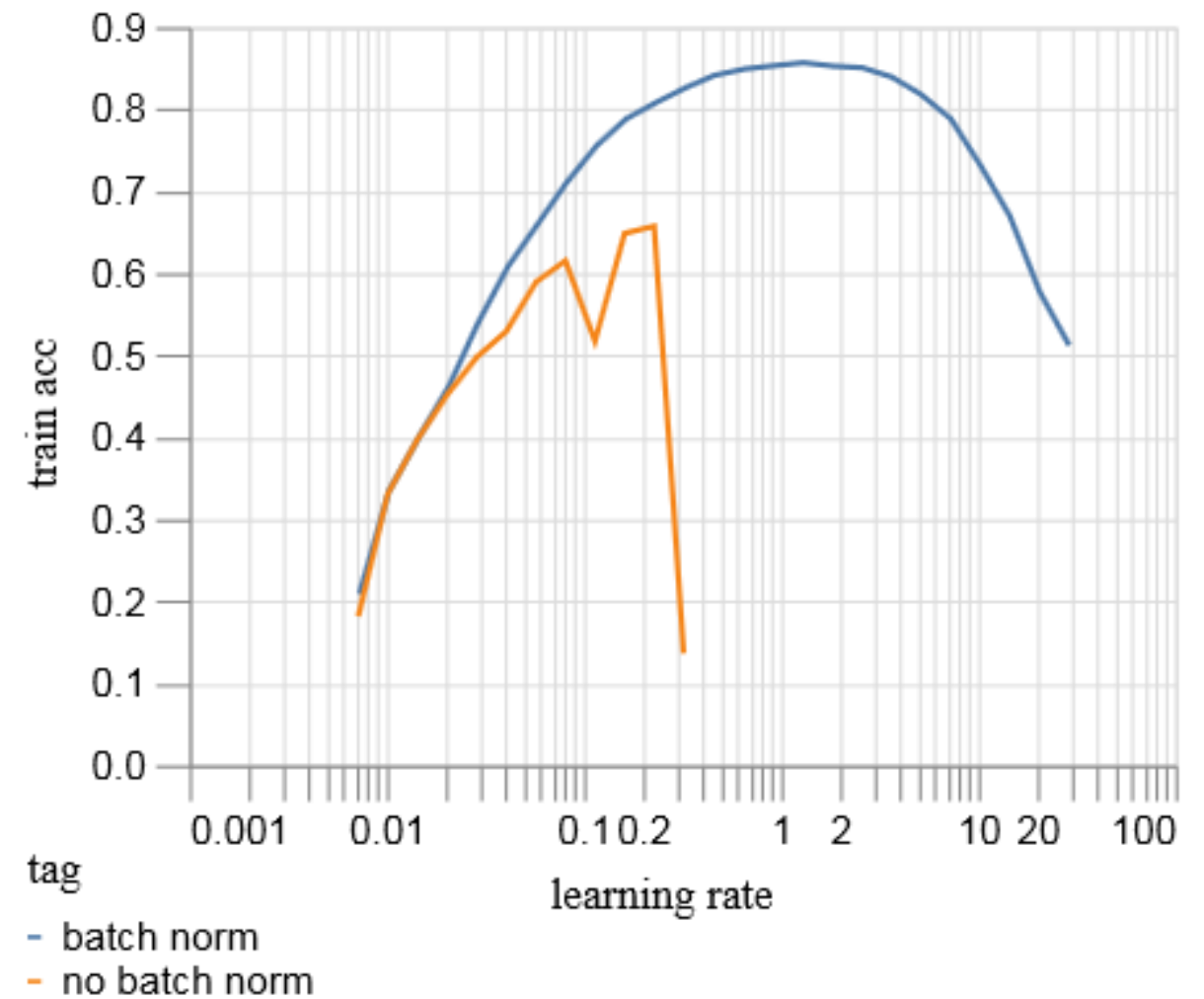
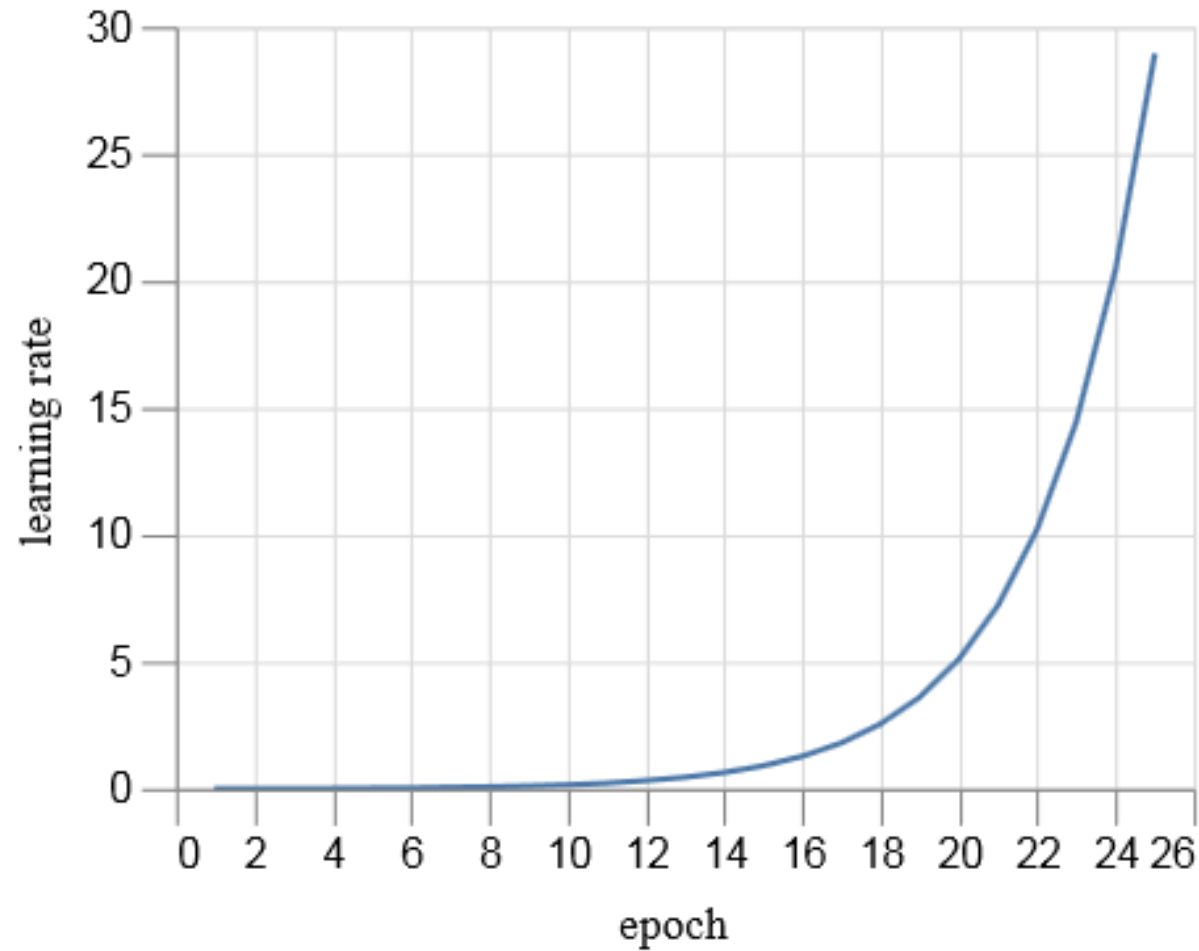
convert batch norm weights back to single precision:
(небольшой хак с реализацией)



Обучение нейронных сетей



Обучение нейронных сетей



Эксперимент, в котором увеличивали темп обучения (слева – как)

<https://myrtle.ai/learn/how-to-train-your-resnet-7-batch-norm/>

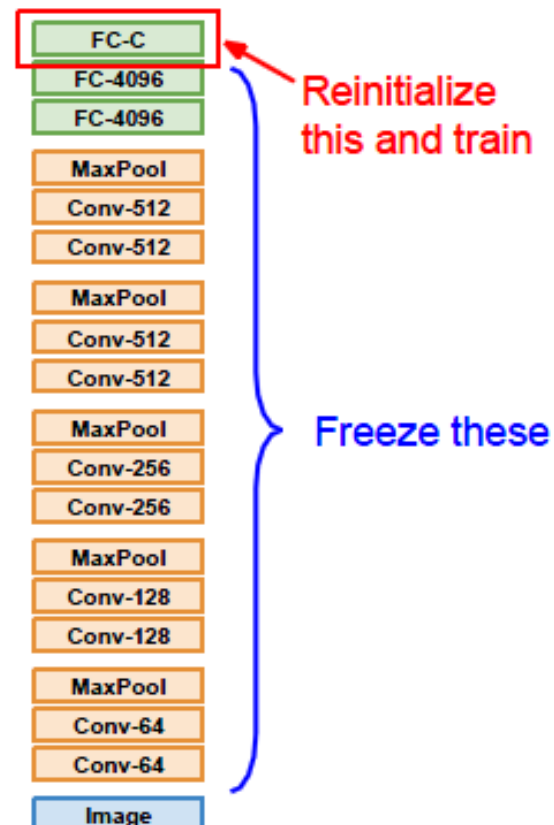
Transfer Learning

Чтобы решать задачи нужны данные...
если данных мало, берём предобученную НС

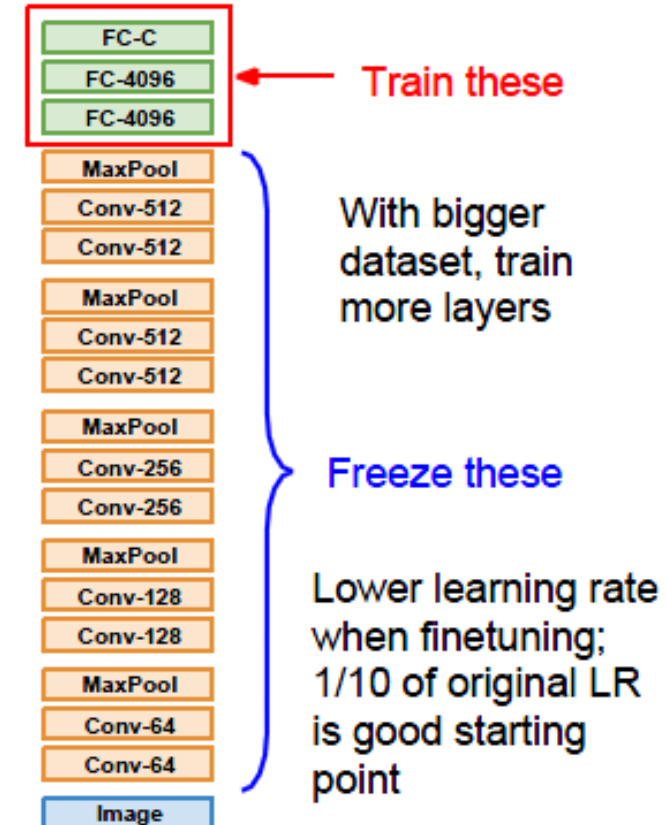
1. Train on Imagenet



2. Small Dataset (C classes)



3. Bigger dataset



Можно не морозить слои, а обучать с меньшим темпом [cs231]

Transfer Learning

Можно решать и другие задачи – для этого перестраивается голова сети
(как бы получаем признаки обученной сетью)

Всю обученную сеть можно:

- полностью переучивать (можно с маленькими темпами на первых слоях)
 - переучивать с какого-то слоя
 - оставить как есть

Transfer Learning

Есть предтренированные НС:

```
from torchvision.models import resnet50, ResNet50_Weights
```

```
# Using pretrained weights:
resnet50(weights=ResNet50_Weights.IMAGENET1K_V1)
resnet50(weights="IMAGENET1K_V1")
resnet50(pretrained=True)  # deprecated
```

```
import torch
```

```
# Option 1: passing weights param as string
model = torch.hub.load("pytorch/vision",
                        "resnet50", weights="IMAGENET1K_V2")
```

```
# Option 2: passing weights param as enum
weights = torch.hub.load("pytorch/vision", "get_weight",
                        weights="ResNet50_Weights.IMAGENET1K_V2")
model = torch.hub.load("pytorch/vision",
                        "resnet50", weights=weights)
```

<https://pytorch.org/hub/>

PYTORCH
HUB

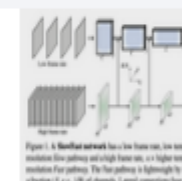
HybridNets 🔗 335

HybridNets - End2End Perception Network



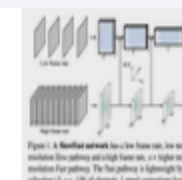
3D ResNet 🔗 2.6k

Resnet Style Video classification networks
pretrained on the Kinetics 400 dataset



SlowFast 🔗 2.6k

SlowFast networks pretrained on the Kinetics
400 dataset



Оптимизация гиперпараметров

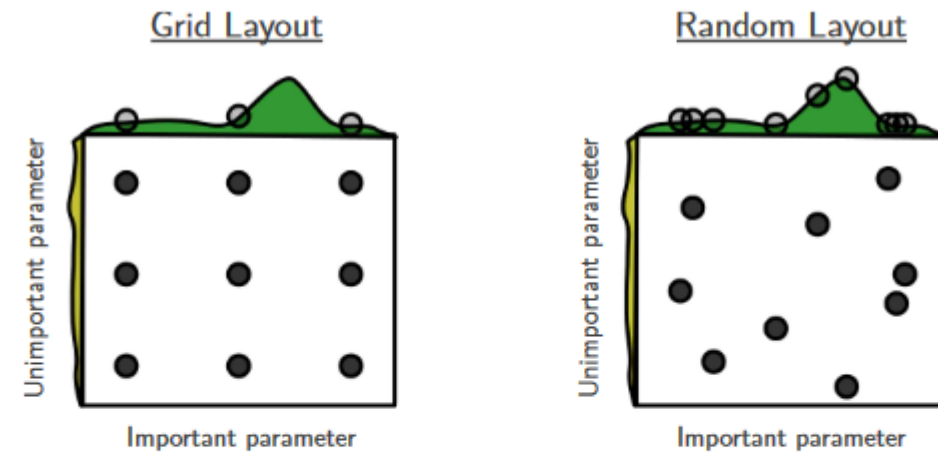
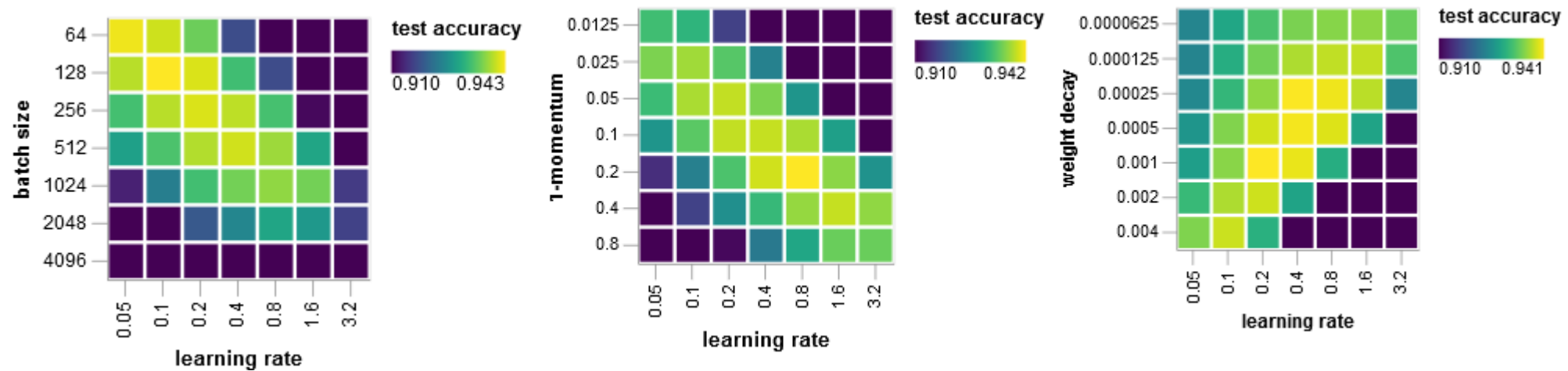


Figure 1: Grid and random search of nine trials for optimizing a function $f(x,y) = g(x) + h(y) \approx g(x)$ with low effective dimensionality. Above each square $g(x)$ is shown in green, and left of each square $h(y)$ is shown in yellow. With grid search, nine trials only test $g(x)$ in three distinct places. With random search, all nine trials explore distinct values of g . This failure of grid search is the rule rather than the exception in high dimensional hyper-parameter optimization.

можно вести более интенсивный поиск в окрестности уже найденного решения

есть другие методы оптимизации!

Оптимизация параметров



можно подойти к поиску оптимальных гиперпараметров как к отдельной задаче оптимизации

покоординатный спуск в пространстве $\left(\frac{\lambda \alpha}{1 - \rho}, \rho, \alpha \right)$

λ – learning rate , ρ – Nesterov momentum , α – weight decay .

<https://myrtle.ai/learn/how-to-train-your-resnet-5-hyperparameters/>

Практические советы

- начинайте с простых архитектур и методов (оптимизации)
- начинайте с небольшого набора данных (для начальных экспериментов, попробуйте переобучиться на батче)
- выбирайте правильную архитектуру (классификация изображений – CNN, последовательности – LSTM/GRU/Transformer и т.п.)
- Добавление параметров $\Rightarrow$ усложнение сети (больше времени на обучение, риск переобучения)
- Используйте средства борьбы с переобучением (см. выше)
- Если данных слишком много средства могут не понадобиться. Если можно – собирайте данные!
- Используйте уже натренированные модели (не геройствуйте – берите проверенное)
- Learning rate часто самый важный параметр – лучше уменьшать (рекомендуют Adam)
- Есть методы настройки параметров лучше, чем structured (grid) search
- Визуализируйте!
- Смотрите на несколько метрик (обязательно на интерпретируемые)
- 0.1 / 1% Rule – веса должны меняться на 1% от своих значений

Практические советы

- Правильная инициализация

ex: при дисбалансе в последнем слое надо так, чтобы выдавалась малая вер-ть

- если что-то не получается можно понизить размерность

по-умному (РСА и т.п.) или просто уменьшить картинки

- если что-то не получается можно упростить / усложнить сеть

- меняйте сеть поэтапно (не вносите более одного изменения)

- осторожней со смешиванием техник (например, dropout и BN)

Li et al. «Understanding the Disharmony between Dropout and Batch Normalization by Variance Shift»

// <https://arxiv.org/abs/1801.05134>

- тренируйте дольше

«One time I accidentally left a model training during the winter break and when I got back in January it was SOTA»

- не доверяйте встроенным программам понижения темпа

Самые важные гиперпараметры

темп обучения
функция ошибки (loss)
размер слоя

инициализация
глубина модели
параметры слоёв (ex: размеры свёрток)
параметр регуляризации

Глупые ошибки

не перевести в режим **train / eval**

забыть **zero_grad()** до **backward()**

сделать softmax для функции, ожидающей логиты (оценки)
nn.CrossEntropyLoss = nn.LogSoftmax + nn.NLLLoss
не нужен softmax

размерности, по которым делаются операции (**nn.Softmax(dim=-1)**)

не создавайте лишнего в forward pass
(например, тогда **nn.Embedding** будет каждый раз новым)

не тестировать с разными параметрами
например, batch_size, т.к. ориентацию матриц можно перепутать,
если матрица квадратная, то ошибки не будет

Глупые ошибки

не оформлять последовательность модулей в

nn.ModuleList / nn.Sequential

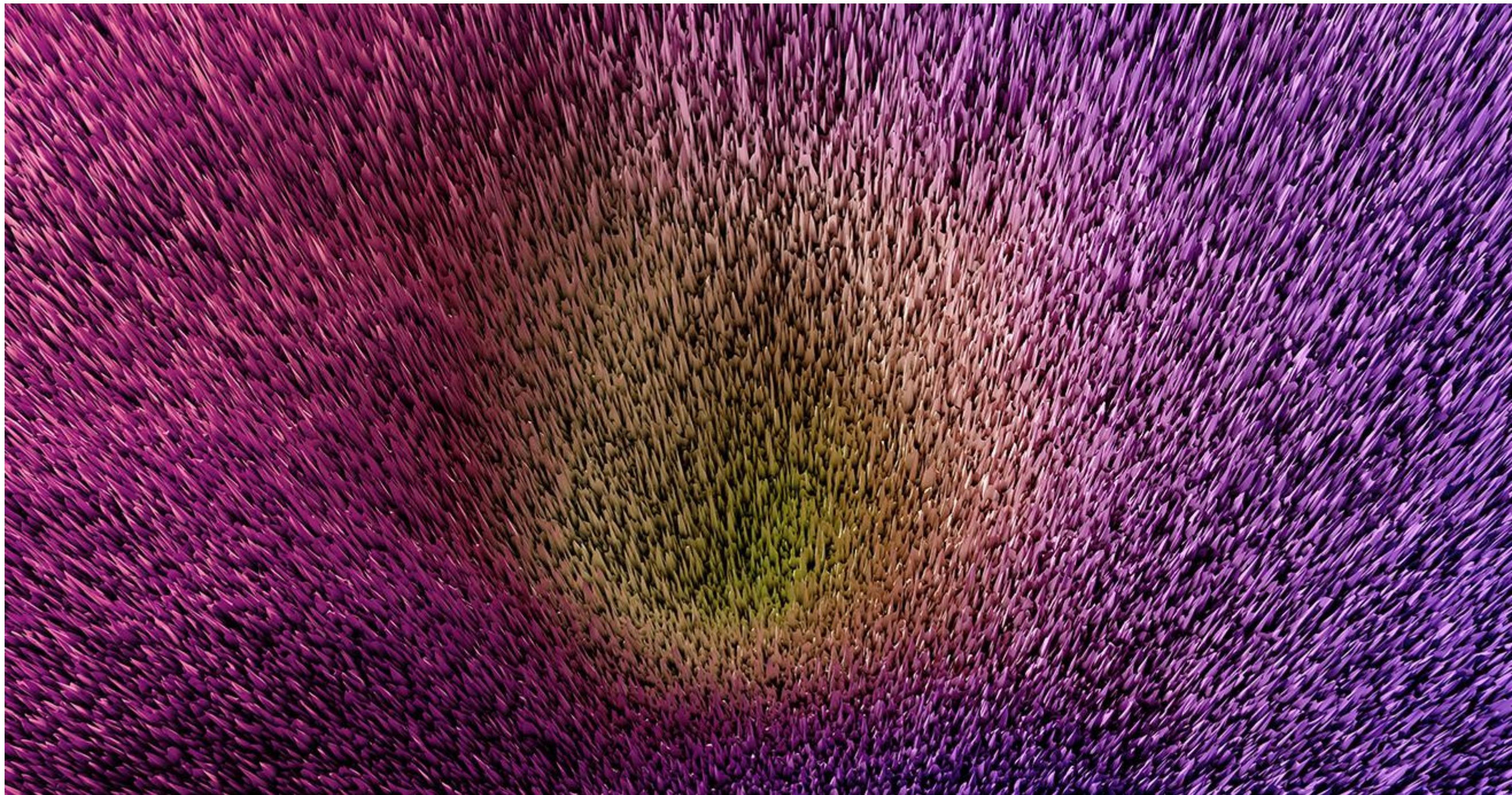
если это внутри другого модуля,

то их параметры автоматически не определятся как параметры обёртки

вызывать **.to(device)** внутри инициализации HC

пренебрегать правильной инициализацией в глубоких сетях





Итог

Много трюков... их число увеличивается
Важны также не отдельные трюки, а комбинации

Иногда «старые трюки» меняются
т.к. не работают в комбинации с новыми,
появляются новые архитектуры,
увеличиваются размеры моделей и датасетов

Ссылки

Советы по настройке сетей

<https://karpathy.github.io/2019/04/25/recipe/>

Debugging in PyTorch

https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/guide3/Debugging_PyTorch.html

Поверхности функций ошибки

<https://losslandscape.com/gallery/>

Трюки для обучения сетей

Tong He et al. «Bag of Tricks for Image Classification with Convolutional Neural Networks» //

<https://arxiv.org/pdf/1812.01187.pdf>